# GPT 模型

## 第 3 章

GPT 模型介绍 · 网络结构 · BPE 编码 · 文本生成

**著者**
邓杭　杨耀东

**教材**
《大语言模型：训练、精调与对齐》

**出版**
清华大学出版社



# 本章目录

3.1 **GPT 模型介绍** — GPT 系列的演进、关键改进与预训练目标 — 第 3 页

3.2 **GPT 网络结构** — 输入层、解码器与输出层 — 第 24 页

3.3 **BPE 编码** — BPE 的词表训练、编码与解码 — 第 48 页

3.4 **GPT 模型生成** — 串行推理与各类生成策略 — 第 60 页

3.5 **小结** — 范式、可扩展性与影响 — 第 73 页

3.6* **附录：语言模型的压缩理论*** — 压缩、编码与似然最大化 — 第 77 页

**阅读建议** 先看 GPT 系列的演进与网络结构（3.1–3.2），再读 BPE 与解码策略（3.3–3.4）。

# 3.1

# GPT 模型介绍

GPT 用纯解码器 Transformer 做自回归预训练：把任务学习转变为预训练学习，并在规模扩展中展现出涌现能力。

本节内容

**3.1.1 GPT 模型发展历史**

3.1.2 GPT 模型关键改进

3.1.3 GPT 预训练建模

# GPT 模型概述

生成式预训练 Transformer（Generative Pre-trained Transformer, GPT）是一种**纯解码器**的 Transformer 架构语言模型，可在通用语料上做自回归预训练。

**通用语料**
无标注文本

→

**自回归预训练**
预测下一个词元

→

**通用模型**
零样本 / 少样本 / 微调

- 随着算力提升，规模越来越大的 GPT 类模型被成功训练，性能随参数规模持续提升。
- 当参数规模增长到特定量级时，模型展现出**涌现**现象：通用任务测评上的性能显著跃升。
- GPT 系列模型是大语言模型发展历程中的一个重要里程碑。

# 生成式预训练的提出

传统机器学习为每个任务收集标注数据、单独训练模型，泛化能力有限、标注成本高昂；生成式预训练改在无标注语料上做自监督学习。

## 任务特定的监督训练

为每个任务收集对应的标注数据，训练专用模型。

训练出的模型难以适应新样本、难以迁移到其他任务。

数据标注成本高昂，难以规模化扩展。

## 生成式预训练 + 微调

在无标签数据集上通过自监督学习（如自回归）训练预训练模型。

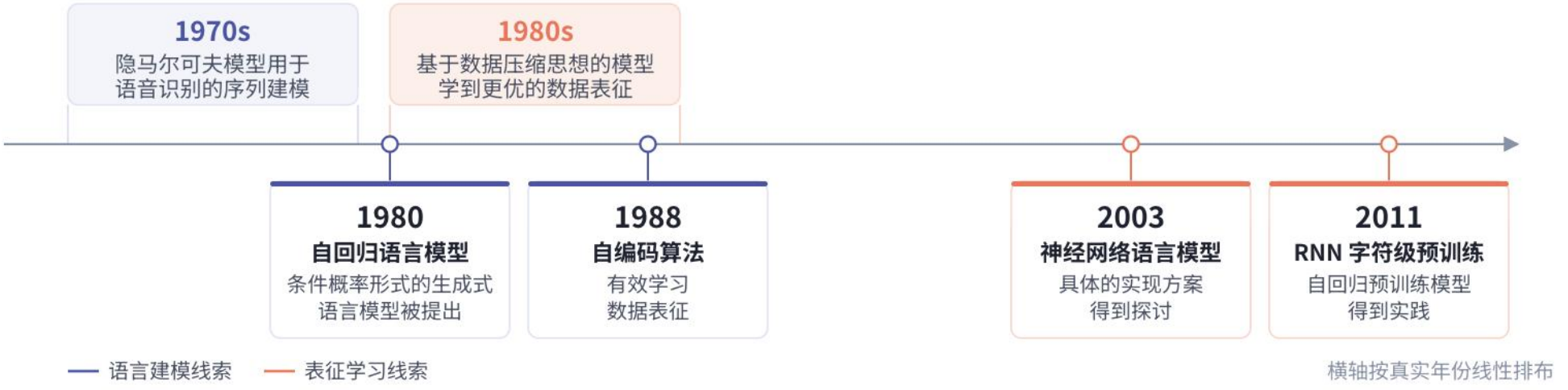
再在有标签的特定数据集上微调，适配下游任务。

避免了任务特定的标注成本；预训练学到的鲁棒特征可迁移到下游任务。

> **理想** 一个鲁棒的系统应当是通用的：能在广泛的领域和任务上运作，无须为每个任务手动创建和标记训练数据集。

# 生成模型的发展脉络

生成模型的研究由来已久，自回归的语言建模与“用压缩学习表征”的思想在数十年前就已出现；两条线索最终在 GPT 上汇合。




- 两条线索并行推进：一边是自回归的语言建模，一边是用压缩换取更优表征。
- 二者在 GPT 上汇合为“自回归目标 + 大规模预训练 + Transformer 架构”。

# GPT-3 的范式转变

GPT-3 及后续版本的诞生标志着人工智能领域一次深刻的范式转变：海量数据与超大规模参数，首次大规模实证了缩放定律。

**175B**

GPT-3 的参数量

**300B**

训练语料的词元数量

**提示词**

无须更新参数即可切换任务

- 仅需提示词——对输入文本增加额外的任务描述或任务示例——单一通用预训练模型就能在大量下游任务中取得卓越表现。
- 其性能甚至可超越为特定任务训练的专项模型，显著改变了依赖监督学习的训练模式。
- 模型随之涌现出复杂推理、代码生成及多语言理解等高级能力，为通往通用人工智能的研究架设了重要桥梁。

# 本章结构

本章从语言建模（自回归预训练的语言表征学习）和模型架构（纯解码器 Transformer）两个角度解析 GPT。

线索一

### 语言建模

自回归地预测下一个词元，用负对数似然训练；预训练损失、困惑度指标刻画模型对文本的概率建模能力。

对应 3.1.3 与 3.4。

线索二

### 纯解码器架构

在 Transformer 的基础上移除编码器与编码器-解码注意力，只保留掩码自注意力、前馈层与前置归一化。

对应 3.2，以 GPT-2 的具体实现为主。

> **说明** 第 2 章已完整描述多头注意力等核心组件，本章重点讨论 GPT-2 在架构上的具体实现与优化细节；附录给出压缩理论视角，帮助理解大语言模型的本质。

模型的应用与后训练细节（微调、强化学习）在第 5、6、8 章讨论。

# 编码器与解码器两条技术路线

Transformer 架构的提出对后续语言模型发展产生了深远影响。翌年，谷歌发布纯编码器的 BERT，OpenAI 推出纯解码器的 GPT-1。

## BERT

基于纯编码器架构（Encoder-Only Transformer）的预训练模型。

双向上下文，适合理解类任务。

与 GPT 同年发布，代表另一条技术路线。

## GPT-1 → GPT-2 → GPT-3

基于纯解码器架构（Decoder-Only Transformer）的预训练模型。

自回归生成，逐步确立预训练架构范式与规模扩展路径。

Instruct-GPT 进一步引入 RLHF，为后训练技术奠定基础。

- OpenAI 陆续发布 GPT-2、GPT-3 和 Instruct-GPT 等迭代版本。
- 关键细节与特性汇总于表 3.1。

# GPT 系列模型

表 3.1　OpenAI 发布的 GPT 系列模型

| 系列 | 模型名称 | 发布时间 | 预训练/后训练 | 主要特性 |
|---|---|---|---|---|
| | GPT-1 | 2018.06 | 是 / 否 | 纯解码器 Transformer |
| | GPT-2 | 2019.02 | 是 / 否 | 自监督训练，最大 1.5B |
| | GPT-3 | 2020.05 | 是 / 否 | 上下文学习，最大 175B |
| | Code-X | 2021.07 | 是 / 否 | 代码继续预训练 |
| | Instruct-GPT | 2022.03 | 否 / 是 | 强化学习（RLHF） |
| ChatGPT (GPT-3.5) | code-davinci-002 | 2022.03 | - / 是 | 代码能力 |
| | text-davinci-002 | 2022.03 | - / 是 | 指令跟随能力 |
| | text-davinci-003 | 2022.09 | - / 是 | RLHF |
| | gpt-3.5-turbo | 2023.03 | - / - | 对话模型 |
| GPT-4 | GPT-4 | 2023.03 | - / - | 强推理能力 |
| | GPT-4 Turbo | 2023.09 | - / - | 长上下文 |
| | GPT-4 Turbo Vision | 2023.09 | - / - | 多模态能力 |
| GPT-4o | GPT-4o-Realtime | 2024.10 | - / - | 实时对话能力 |
| | GPT-4o | 2024.05 | - / - | 发布版本 |
| | ChatGPT-4o | 2024.08 | - / - | 上线 ChatGPT 版本 |
| o1 | o1-mini | 2024.09 | - / - | 强推理能力 |
| | o1 | 2024.12 | - / - | 强推理能力 |
| | o1-pro | 2025.03 | - / - | 强推理能力 |
| o3 | o3-mini | 2025.01 | - / - | 强推理模型 |
| gpt-oss | gpt-oss-20B | 2025.08 | - / - | 开源 MoE 架构 |
| | gpt-oss-120B | 2025.08 | - / - | 开源 MoE 架构 |
| GPT-5 | GPT-5 | 2025.08 | - / - | 强推理模型 |

“-”表示该信息未公开；“预训练/后训练”仅以已发表文献标记是否做对应训练；GPT-3.5 系列通过网页或 API 使用。

**依据** 《大语言模型：训练、精调与对齐》第 3.1.1 节（表 3.1）

# GPT 系列的能力演进

GPT-2

### 零样本

未经微调的预训练模型直接用于下游任务。零样本任务指训练阶段未接触过任何相关样本，输入上也没有示例。

对预训练模型的泛化性能要求较高。

GPT-3

### 上下文学习

不更新参数，只在输入中增加结构化示例，即可提升具体任务性能。

175B 参数、300B 词元训练，使模式识别在提示中生效。

Instruct-GPT

### 人类反馈对齐

基于人类反馈的强化学习（RLHF），显著提升指令理解能力与人类偏好对齐度。

推动大语言模型走向实用化和商业化的关键技术。

- 语言预训练模型的核心目标并非直接解决特定自然语言处理任务，而是获取丰富且鲁棒的语言数据本身的表征。
- 此类模型通常仅需在特定下游任务数据上微调，即可高效适配。

**依据** 《大语言模型：训练、精调与对齐》第 3.1.1 节

# GPT-1 的模型结构

相较于包含编码器和解码器的原始 Transformer，GPT-1 只保留解码器，参数量显著减少，输入形式也更加灵活。

- 移除编码器，因此不再需要“编码器输入”与“解码器输入”两套序列。
- 移除编码器-解码注意力模块，解码块内只剩掩码自注意力与前馈层。
- 输入输出形式统一：一段词元序列进，下一个词元的分布出。

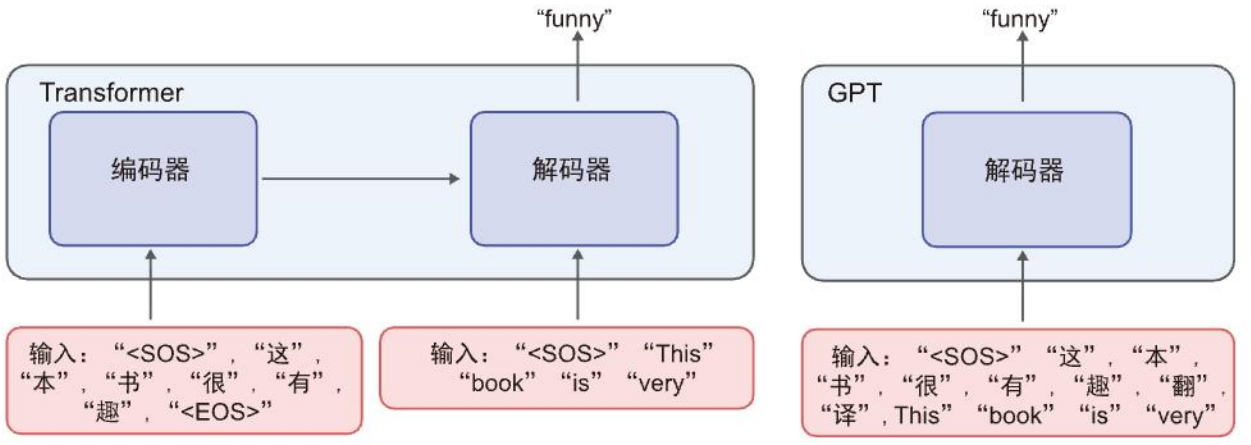



图 3.1(a)　Transformer　　图 3.1(b)　GPT

**图源** 《大语言模型：训练、精调与对齐》图 3.1

# GPT-1 的两阶段训练

先在大规模无标签文本上预训练，学习通用的语言表示；再在特定下游任务的有标签数据上做有监督微调。




- 微调时会在预训练模型顶部添加一个与特定任务相关的输出层（例如线性分类层），见图 3.2(a)。
- 在微调过程中引入预训练损失，能够提升监督模型的泛化性并加快收敛（见 3.1.3 的式 (3-2)）。
- 分词器方面，GPT-1 采用改进的 BPE 分词器，分词更加灵活和通用，详见 3.3 节；后续语言模型均沿用 BPE 方案。

# GPT-2 的零样本能力

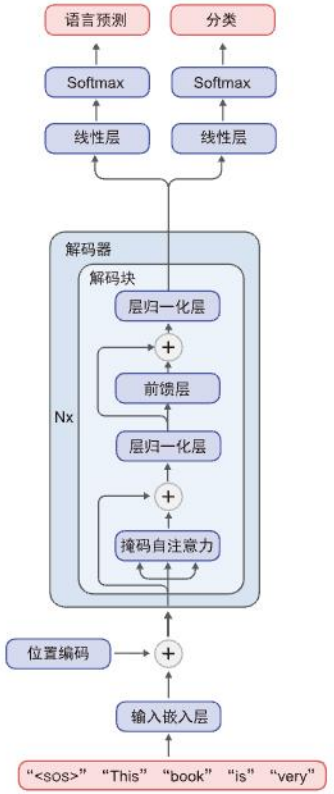



图 3.2(a)　GPT-1

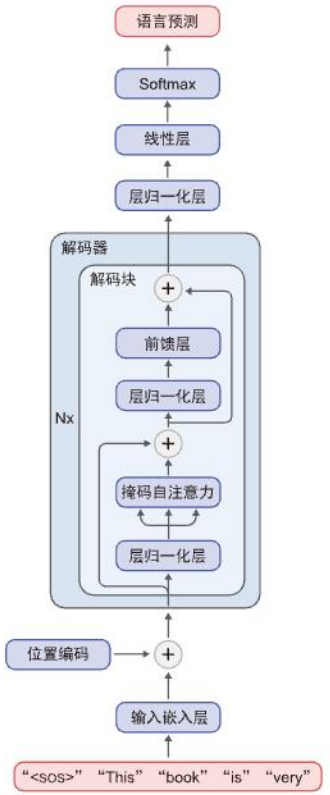



图 3.2(b)　GPT-2

GPT-2 延续纯解码器结构与自回归语言建模目标，并发布四种规模的版本，最大的 GPT-2 XL 达到 15 亿（1.5B）参数。

- 核心突破是具有强大的零样本学习能力：无须在任何特定任务的监督数据上微调，即可直接应用于问答、翻译、摘要等任务。
- 与 GPT-1 不同，GPT-2 移除了下游任务微调阶段及其对应的监督任务输出，只保留语言预测头。
- 图 (b) 相较 (a) 少了一条分类分支：输出端只剩语言预测头。
- 文本生成时通常使用 Top-$K$ 采样策略以提高输出多样性，详见 3.4.2 节。

**图源** 《大语言模型：训练、精调与对齐》图 3.2

# GPT-3 的上下文学习

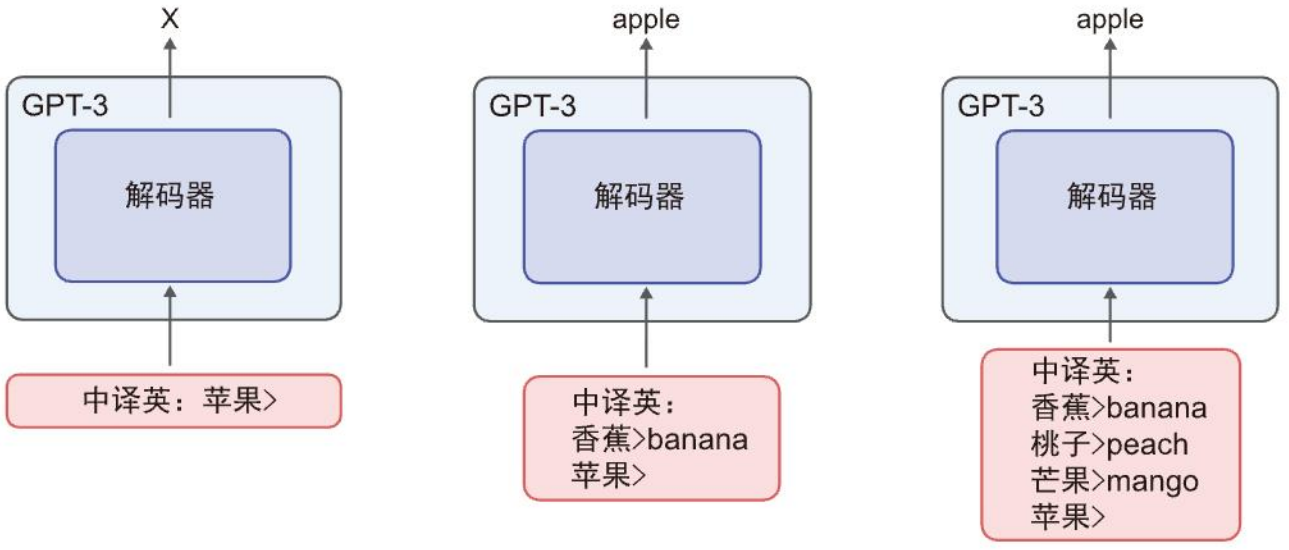



图 3.3(a)　零样本　　图 3.3(b)　单样本　　图 3.3(c)　少样本

图 3.3　GPT-3 上下文学习示例（中译英：苹果 → apple）

- GPT-3 把参数量扩展到 175B，在 300B 词元的通用语料上训练。
- 核心创新是上下文学习：无须微调，只需在输入提示中添加任务示例。
- (a) 零样本未能生成正确翻译；(b) 单样本、(c) 少样本成功生成正确翻译。
- 在输入里添加示例，使模型通过模式识别提升任务表现。

**图源** 《大语言模型：训练、精调与对齐》图 3.3

# InstructGPT 的三阶段训练

Instruct-GPT 采用基于人类反馈的强化学习（RLHF）算法，提升模型的指令理解与输出偏好性。

**阶段 1**

有监督微调

用示范数据教模型跟随指令

→

**阶段 2**

奖励模型训练

学习人类对输出的偏好

→

**阶段 3**

强化学习优化

按奖励信号调整策略

- 三个阶段中，后两者为强化学习相关的训练阶段。
- RLHF 的详细技术细节在第 8.3 节讨论。
- 这一范式使模型从“会续写”走向“听得懂指令、符合人类偏好”。

# 预训练目标

GPT 是一种自回归语言模型，核心预训练任务是预测给定输入序列之后的下一个词元（Next Token Prediction）。

$$\mathcal{L}_{\text{pretrain}} = -\sum_{t=1}^{n} \log p_\theta(x_t | \boldsymbol{x}_{<t}) \tag{3-1}$$

输入序列 $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)$；$\boldsymbol{x}_{<t} = (x_1, \ldots, x_{t-1})$ 表示位置 $t$ 之前的所有词元序列；$p_\theta(x_t | \boldsymbol{x}_{<t})$ 是给定历史预测下一个词元的概率。

- 训练时通过最小化负对数似然优化参数 $\theta$，损失函数用交叉熵衡量预测分布与真实词元的差异。
- 对于一个长度为 $n$ 的词元序列，解码器会**并行**执行 $n$ 个预测任务，达到高效预训练的目的。

# GPT-1 与 GPT-2 的输出头差异

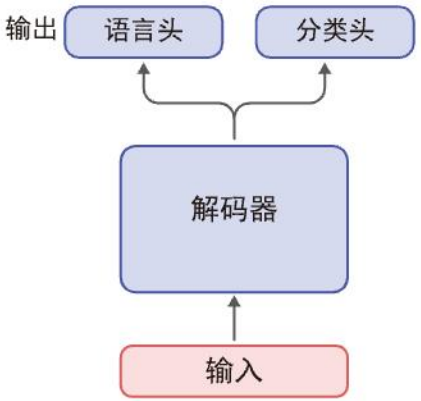



图 3.4(a)　GPT-1

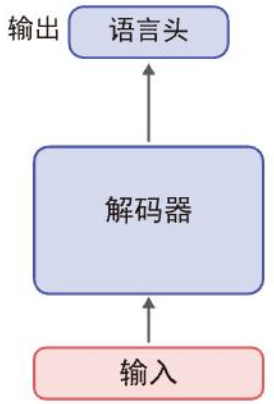



图 3.4(b)　GPT-2

图 3.4　GPT-1 和 GPT-2 输出头对比（微调阶段是否保留分类头）

- GPT-1 在预训练的语言头之外，为下游任务再添加一个分类头，微调时两个头一起使用。
- GPT-2 只保留语言头，把所有任务都改写为序列续写。
- 输出头的取舍，直接对应下面两种训练目标。

**图源** 《大语言模型：训练、精调与对齐》图 3.4

# GPT-1 的两阶段损失

GPT-1 的训练分为两阶段：先无监督预训练，损失为 $\mathcal{L}_{\text{pretrain}}$；再把预训练好的模型用于下游任务，进行有监督微调。

$$\mathcal{L}_{\text{GPT-1}} = \mathcal{L}_{\text{pretrain}} + \lambda \mathcal{L}_{\text{task}} \tag{3-2}$$

$\lambda$ 是一个可选的任务损失权重系数（有时可设为 1）。

$$\mathcal{L}_{\text{task}} = -\log p_\theta(y|\boldsymbol{x}, \texttt{task}) \tag{3-3}$$

$y$ 是输入序列 $\boldsymbol{x}$ 对应的真实类别标签；`task` 为任务描述文本，与输入序列拼接成结构化输入。

- 例如情感分类：情感分类：环境很好 <CLS>；模型处理完整个输入后，取 <CLS> 标记或序列末尾位置的表示向量，经分类头计算 $p_\theta(y|\boldsymbol{x}, \texttt{task})$。
- 类别标签如“积极”“消极”；引入预训练损失可提升泛化性并加快收敛。

**依据** 《大语言模型：训练、精调与对齐》第 3.1.3 节（式 (3-2)、(3-3)）

# GPT-2 的单一训练目标

GPT-2 摒弃了 GPT-1 中针对下游任务添加任务特定损失 $\mathcal{L}_{\text{task}}$ 和输出层的微调范式，只探索预训练的语言建模目标。

$$\mathcal{L}_{\text{GPT-2}} = \mathcal{L}_{\text{pretrain}} \tag{3-4}$$

训练目标与式 (3-1) 完全一致：逐位置预测下一个词元。

**示例** 提示模板："情感分类: 这里的环境很好。情感是:"——把分类任务改写成序列续写，期望模型生成代表类别的词元（如"积极"）。

- GPT-2 灵活的输入输出形式，无须针对特定任务类别设计任务头（任务头通常也需要训练）。
- 同一个预训练模型因此能处理通用的语言任务。

# WebText 数据集

GPT-2 的训练数据主要来源于海量互联网文本，即 WebText 数据集：通过网络爬取高质量文章和网页构建。

**40GB**
语料总计

**800 万**
文档数量

**50 257**
BPE 词表大小

- 数据经过预处理（如去除 HTML 标签和内容过滤），以确保语料的多样性和质量。
- 数据量通常以分词器处理原始文本后得到的**词元量**来衡量。
- 预训练时需要把词元序列按预设长度切分为多个数据块，长度受模型最大上下文长度和计算资源限制。

> **例** 一本小说转换为 10 000 个词元，切分长度 512，则划分为 $\lceil 10000/512 \rceil = 20$ 个块；第 20 块长度不足，需填充 <PAD> 使所有样本块等长。

# 困惑度的定义

在自回归语言模型训练中，困惑度（Perplexity, PPL）是评估模型预测能力的关键指标，衡量模型对给定文本序列进行概率建模的流畅性。

$$H(\theta, \boldsymbol{x}) = -\frac{1}{n}\sum_{t=1}^{n}\log\left(p_\theta(x_t|\boldsymbol{x}_{<t})\right) \tag{3-5}$$

$H(\theta, \boldsymbol{x})$ 是模型 $\theta$ 在序列 $\boldsymbol{x}$ 上的交叉熵，量化模型对真实样本分布估计的平均不确定性。

$$\mathrm{PPL}(\theta, \boldsymbol{x}) = e^{H(\theta, \boldsymbol{x})} \tag{3-6}$$

困惑度是平均交叉熵的指数：PPL 越低，预测越准确、不确定性越低；越高则不确定性越高。

在测试集上计算的困惑度能够反映模型对未见数据的泛化能力。

**依据** 《大语言模型：训练、精调与对齐》第 3.1.3 节（式 (3-5)、(3-6)）

# 模型规模与困惑度的关系

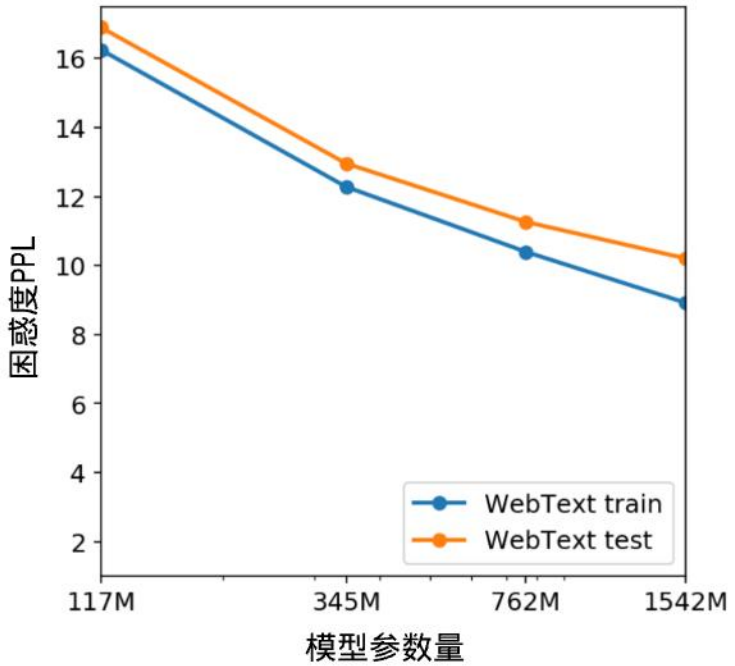



图 3.5　GPT-2 各尺寸模型困惑度测评结果

- 图中对比不同规模的 GPT-2 模型在 WebText 训练集和测试集上的困惑度。
- 随着模型规模增大，困惑度显著降低。
- 测试集上的困惑度反映模型对未见数据的泛化能力。

> **注**　困惑度主要从统计层面评估建模性能；要评估知识、自然语言理解、机器翻译、数学推理等能力，还需更广泛的评测基准，如涌现现象研究系统整理的测评任务。

**图源** 《大语言模型：训练、精调与对齐》图 3.5

# 3.2

# GPT 网络结构

本节内容

**3.2.1 输入层**

3.2.2 解码器

3.2.3 输出层

纯解码器 Transformer 天然适合自回归训练：注意力机制高度并行，无须依赖上一时间步的隐状态。以 GPT-2 为主讲解。

# GPT 的三个组成部分

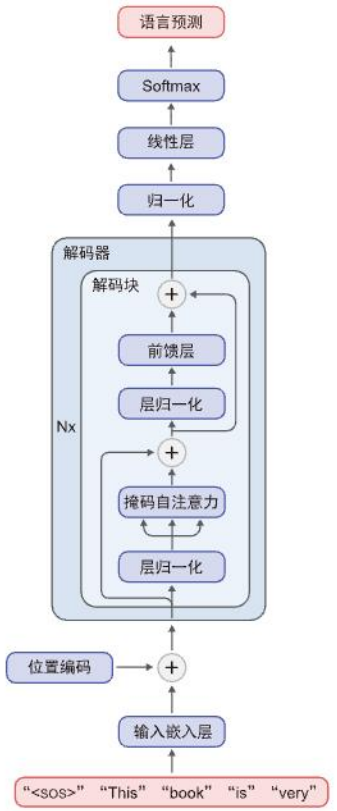



图 3.6　GPT-2 网络结构示意图

**第 1 部分**

## 输入层

与 Transformer 的输入层一致：词嵌入层加位置编码层。

**第 2 部分**

## 解码器

由多个解码块堆叠而成，每个块含前置层归一化、掩码自注意力与前馈层。

**第 3 部分**

## 输出层

解码器采用前置层归一化，故输出层先经一个归一化层，再由线性层投影到词表维度，最后 Softmax 得到概率分布。

**图源** 《大语言模型：训练、精调与对齐》图 3.6

# GPT 的四种模型规格

GPT-2 系列模型具有 4 种规格，模型的参数量取决于词嵌入向量维度和解码器层数，并随着这两者的增加而显著增大。

表 3.2　GPT-2 系列模型架构参数

| 模型名称 | 词嵌入维度 | 解码块层数 | 参数量 | 约合（亿） |
|---|---|---|---|---|
| GPT-2 | 768 | 12 | 117M | 1.17 |
| GPT-2 Medium | 1024 | 24 | 345M | 3.45 |
| GPT-2 Large | 1280 | 36 | 762M | 7.62 |
| GPT-2 XL | 1600 | 48 | **1542M** | **15.42** |

- 维度每提高一档，层数同时加深，参数量近似按平方增长：768→1600 维度、12→48 层，参数量 117M→1542M。
- 表里的 M 即百万（million），$1542\mathrm{M} = 1.542 \times 10^9$ 个参数，即约 15 亿参数规模，也记作 **1.5B**：$B$ 表示十亿（billion），$1.5\mathrm{B} = 1.5 \times 10^9$，与 1542M 是同一个量。

> **换算**　亿 $= 10^8$，M（million）$= 10^6$，B（billion）$= 10^9$；故 $1542\mathrm{M} = 1.542 \times 10^9 \approx 1.5\mathrm{B}$，即 15 亿。同理 7B 为 70 亿参数、70B 为 700 亿参数。

**依据**　《大语言模型：训练、精调与对齐》第 3.2 节（表 3.2）

# 输入层

GPT-2 采用 Transformer 解码器的输入层结构。由于仅包含解码器，其输入层即为解码器的输入层，包含词嵌入层和位置编码层。

**词元编号 $\boldsymbol{x}$**

$(x_1, x_2, \cdots, x_n)$

→

**查嵌入矩阵 $\boldsymbol{E}$**

$\boldsymbol{x}_i = \boldsymbol{E}_{x_i} \in \mathbb{R}^{1\times d}$

→

**加位置编码**

$\boldsymbol{x}'_i = \boldsymbol{x}_i + \mathrm{PE}_i$

→

$\boldsymbol{X} \in \mathbb{R}^{n\times d}$

送入解码器

$$\boldsymbol{x}'_i = \boldsymbol{x}_i + \mathrm{PE}_i \tag{3-7}$$

$\boldsymbol{E} \in \mathbb{R}^{|\mathcal{V}|\times d}$ 为可学习的嵌入矩阵，$\boldsymbol{x}_i = \boldsymbol{E}_{x_i}$ 即 $\boldsymbol{E}$ 的第 $x_i$ 行；$\mathrm{PE}_i$ 为位置 $i$ 的位置编码向量。

- 词表 $\mathcal{V}$ 来自在通用语料上训练的 BPE 分词器，细节见 3.3 节。
- 整个输入序列得到矩阵 $\boldsymbol{X} \in \mathbb{R}^{n\times d}$，第 $i$ 行即 $\boldsymbol{x}'_i$。

# 解码器结构

GPT-2 的解码器由 $L$ 个相同结构的解码块堆叠而成。相较于原始 Transformer 解码器，GPT-2 移除了编码器-解码器注意力层。

$$\boldsymbol{X}^{(l)} = \texttt{Decoder_Block}_l(\boldsymbol{X}^{(l-1)}), \quad l = 1, 2, \cdots, L \tag{3-8}$$

$\boldsymbol{X}^{(l-1)}$ 为第 $l-1$ 层的输出（$\boldsymbol{X}^{(0)}$ 为解码器输入），$\texttt{Decoder_Block}_l(\cdot)$ 表示第 $l$ 层的解码块。

## 原始 Transformer 解码块

掩码自注意力 + 编码器-解码注意力 + 前馈层。

需要编码器输出 $\boldsymbol{X}_{\text{enc}}$ 作为 $\boldsymbol{K}, \boldsymbol{V}$ 的来源。

## GPT-2 解码块

层归一化、掩码自注意力、前馈网络，每个组件后均接残差连接。

采用前置层归一化，且仅包含掩码自注意力层。

# 解码块的组成

$$\boldsymbol{X}_{\text{ln1}} = \texttt{Layer_Norm_1}(\boldsymbol{X}^{(l-1)}) \tag{3-9}$$

$$\boldsymbol{X}_{\text{attn}} = \texttt{Multi_Head_Attention}(\boldsymbol{X}_{\text{ln1}}, \boldsymbol{M}) \tag{3-10}$$

$$\boldsymbol{X} = \boldsymbol{X}_{\text{attn}} + \boldsymbol{X}^{(l-1)} \qquad \text{(残差连接1)} \tag{3-11}$$

$$\boldsymbol{X}_{\text{ln2}} = \texttt{Layer_Norm_2}(\boldsymbol{X}) \tag{3-12}$$

$$\boldsymbol{X}_{\text{ffn}} = \texttt{Feed_Forward}(\boldsymbol{X}_{\text{ln2}}) \tag{3-13}$$

$$\boldsymbol{X}^{(l)} = \boldsymbol{X}_{\text{ffn}} + \boldsymbol{X} \qquad \text{(残差连接2)} \tag{3-14}$$

式 (3-9)–(3-11) 为掩码自注意力子层，式 (3-12)–(3-14) 为前馈子层；$\boldsymbol{X}_{\text{attn}} + \boldsymbol{X}^{(l-1)}$ 即残差连接。

- 归一化放在子层**之前**（前置归一化），残差通路上始终是未归一化的 $\boldsymbol{X}^{(l-1)}$ 与 $\boldsymbol{X}$。
- 两个层归一化的参数相互独立；$\boldsymbol{M}$ 为下三角掩码矩阵。

**依据** 《大语言模型：训练、精调与对齐》第 3.2.2 节（式 (3-9)–(3-14)）

# 层归一化

解码块通常包含两个参数相互独立的层归一化层。其核心思想是独立地对每个词元的特征向量 $\boldsymbol{x}_i \in \mathbb{R}^d$ 做归一化。

$$\mu_i = \frac{1}{d}\sum_{j=1}^{d} x_{ij} \tag{3-15}$$

$$\sigma_i^2 = \frac{1}{d}\sum_{j=1}^{d}(x_{ij} - \mu_i)^2 \tag{3-16}$$

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \tag{3-17}$$

$$\tilde{x}_{ij} = \gamma_j \hat{x}_{ij} + \beta_j \tag{3-18}$$

$x_{ij}$ 为第 $i$ 个词元的第 $j$ 维特征；$\mu_i, \sigma_i^2$ 为样本 $i$ 的特征均值与方差；$\epsilon > 0$ 防止除零（通常 $10^{-5}$ 量级）。

**优点**

归一化只依赖当前样本自身的统计量，不依赖同批次其他样本，天然适应批次内长度不一的序列。

**参数**

$\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ 为可学习的缩放与平移向量，默认初始化 $\boldsymbol{\gamma} = \mathbf{1}$、$\boldsymbol{\beta} = \mathbf{0}$。

**依据** 《大语言模型：训练、精调与对齐》第 3.2.2 节（式 (3-15)–(3-18)）

# 后置归一化的问题

GPT-2 采用前置层归一化代替后置层归一化。先看后置归一化的展开形式：对残差连接后的输出做归一化。

$$\boldsymbol{X}^{(l+1)} = \texttt{Norm}(\boldsymbol{X}^{(l)} + \mathcal{F}_l(\boldsymbol{X}^{(l)})) \tag{3-19}$$

$$= \frac{\boldsymbol{X}^{(l)} + \mathcal{F}_l(\boldsymbol{X}^{(l)})}{\sqrt{2}} \tag{3-20}$$

$\mathcal{F}_l(\cdot)$ 是包含训练参数的第 $l$ 层网络层（注意力函数或前馈层函数）。$\boldsymbol{X}^{(l)}$ 服从均值 0、方差 1 的分布，经残差相加后方差为 2，除以标准差 $\sqrt{2}$ 后 $\boldsymbol{X}^{(l+1)}$ 的均值回到 0、方差回到 1。

把每一层写开，得到逐层的归一化式：

$$\boldsymbol{X}^{(l+1)} = \texttt{Norm}(\boldsymbol{X}^{(l)} + \mathcal{F}_l(\boldsymbol{X}^{(l)})) \tag{3-21}$$

$$\boldsymbol{X}^{(l)} = \texttt{Norm}(\boldsymbol{X}^{(l-1)} + \mathcal{F}_{l-1}(\boldsymbol{X}^{(l-1)})) \tag{3-22}$$

$$\boldsymbol{X}^{(2)} = \texttt{Norm}(\boldsymbol{X}^{(1)} + \mathcal{F}_1(\boldsymbol{X}^{(1)})) \tag{3-23}$$

**依据** 《大语言模型：训练、精调与对齐》第 3.2.2 节（式 (3-19)–(3-23)）

# 后置归一化的信息衰减

$$\boldsymbol{X}^{(l+1)} = \texttt{Norm}(\boldsymbol{X}^{(l)} + \mathcal{F}_l(\boldsymbol{X}^{(l)})) \tag{3-24}$$

$$= \frac{\boldsymbol{X}^{(l)}}{\sqrt{2}} + \frac{\mathcal{F}_l(\boldsymbol{X}^{(l)})}{\sqrt{2}} \tag{3-25}$$

$$= \frac{\boldsymbol{X}^{(l-1)} + \mathcal{F}_{l-1}(\boldsymbol{X}^{(l-1)})}{(\sqrt{2})^2} + \frac{\mathcal{F}_l(\boldsymbol{X}^{(l)})}{\sqrt{2}} \tag{3-26}$$

$$= \frac{\boldsymbol{X}^{(l-1)}}{(\sqrt{2})^2} + \frac{\mathcal{F}_{l-1}(\boldsymbol{X}^{(l-1)})}{(\sqrt{2})^2} + \frac{\mathcal{F}_l(\boldsymbol{X}^{(l)})}{\sqrt{2}} \tag{3-27}$$

$$= \frac{\boldsymbol{X}^{(1)}}{(\sqrt{2})^l} + \frac{\mathcal{F}_1(\boldsymbol{X}^{(1)})}{(\sqrt{2})^l} + \cdots + \frac{\mathcal{F}_l(\boldsymbol{X}^{(l)})}{\sqrt{2}} \tag{3-28}$$

逐步代入式 (3-21) 展开：每一层的残差项都再被除以一次 $\sqrt{2}$。

- 随网络深度增加，第一项 $\dfrac{\boldsymbol{X}^{(1)}}{(\sqrt{2})^l}$ 迅速变小，原始信息 $\boldsymbol{X}^{(1)}$ 被严重削弱，残差连接未起到作用。
- 预训练初期参数随机初始化，标准后置归一化的 Transformer 梯度不稳定，常需学习率预热来缓解。

**依据** 《大语言模型：训练、精调与对齐》第 3.2.2 节（式 (3-24)–(3-28)）

# 前置归一化

$$\boldsymbol{X}^{(l+1)} = \boldsymbol{X}^{(l)} + \mathcal{F}_l(\texttt{Norm}(\boldsymbol{X}^{(l)})) \tag{3-29}$$

$$\boldsymbol{X}^{(l)} = \boldsymbol{X}^{(l-1)} + \mathcal{F}_{l-1}(\texttt{Norm}(\boldsymbol{X}^{(l-1)})) \tag{3-30}$$

$$\boldsymbol{X}^{(2)} = \boldsymbol{X}^{(1)} + \mathcal{F}_1(\texttt{Norm}(\boldsymbol{X}^{(1)})) \tag{3-31}$$

对输入数据先归一化，残差通路直接相加，不经过任何缩放。

对 $\boldsymbol{X}^{(l+1)}$ 逐层展开：

$$\begin{aligned}
\boldsymbol{X}^{(l+1)} &= \boldsymbol{X}^{(l)} + \mathcal{F}_l(\texttt{Norm}(\boldsymbol{X}^{(l)})) && (3\text{-}32)\\
&= \boldsymbol{X}^{(l-1)} + \mathcal{F}_l(\texttt{Norm}(\boldsymbol{X}^{(l-1)})) + \mathcal{F}_{l-1}(\texttt{Norm}(\boldsymbol{X}^{(l)})) && (3\text{-}33)\\
&= \boldsymbol{X}^{(1)} + \mathcal{F}_1(\texttt{Norm}(\boldsymbol{X}^{(1)})) + \cdots + \mathcal{F}_l(\texttt{Norm}(\boldsymbol{X}^{(l)})) && (3\text{-}34)\\
&= \boldsymbol{X}^{(1)} + \mathcal{F}_1(\boldsymbol{X}^{(1)}/\sqrt{1}) + \cdots + \mathcal{F}_l(\boldsymbol{X}^{(l)}/\sqrt{l}) && (3\text{-}35)
\end{aligned}$$

**依据** 《大语言模型：训练、精调与对齐》第 3.2.2 节（式 (3-29)–(3-35)）

# 前置归一化的优势

## 后置归一化（Post-Norm）

$\boldsymbol{X}^{(l+1)} = \texttt{Norm}(\boldsymbol{X}^{(l)} + \mathcal{F}_l(\boldsymbol{X}^{(l)}))$。

原始信息 $\boldsymbol{X}^{(1)}$ 被 $(\sqrt{2})^l$ 不断稀释，深层残差失效。

训练初期梯度不稳定，需要学习率预热。

## 前置归一化（Pre-Norm）

$\boldsymbol{X}^{(l+1)} = \boldsymbol{X}^{(l)} + \mathcal{F}_l(\texttt{Norm}(\boldsymbol{X}^{(l)}))$。

每层的 $\boldsymbol{X}^{(1)}$ 分量保持一致，残差通路始终畅通。

训练时梯度更加稳定，不需要预热。

- 前置归一化下，随着深度增加，网络越深其输出变化越小，即 $\boldsymbol{X}^{(l+1)} \approx \boldsymbol{X}^{(l+2)}$，深层网络的输出分布不会有太大变化。
- 代价是解码器输出 $\boldsymbol{X}^{(l+1)}$ 的标准差不再为 1，因此需要在解码器后**额外增加一个层归一化层**，把输出标准差变换为 1。
- 前置归一化因此成为当前大语言模型主流的网络结构设计。

**依据** 《大语言模型：训练、精调与对齐》第 3.2.2 节

# 掩码自注意力

掩码自注意力是自注意力模块的变体，核心区别在于掩码的应用：用下三角掩码矩阵把未来位置的注意力分数设为负无穷，避免信息泄露。

## 标准自注意力

允许访问序列中所有位置的信息，获取双向上下文。

适合编码器与理解类任务。

## 掩码自注意力

只利用历史信息，阻止模型预测当前词元时看到后续位置。

适用于自回归生成；仅利用历史信息建模通常认为更具挑战性。

- 实现上常把查询、键、值的投影权重合并为一个矩阵，一次矩阵乘法算出三份投影，利于 GPU 并行。
- 随后沿特征维度切分为 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$，再各自拆分为 $h$ 个头并行计算。
- 最后把各头特征拼接，经输出投影层得到注意力输出。

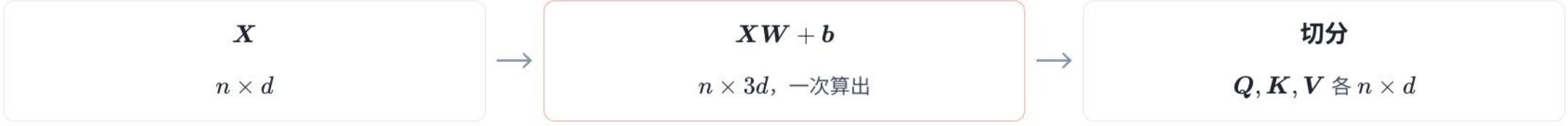
# Q、K、V 的合并投影

$$\boldsymbol{W} \leftarrow [\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v] \in \mathbb{R}^{d \times 3d}, \quad \boldsymbol{b} \in \mathbb{R}^{3d} \tag{3-36}$$

$$\boldsymbol{X}' = \boldsymbol{X}\boldsymbol{W} + \boldsymbol{b} \quad \in \mathbb{R}^{n \times 3d} \tag{3-37}$$

$$\boldsymbol{X}' \rightarrow [\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}] \quad \text{其中} \quad \boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{n \times d} \tag{3-38}$$

式 (3-36) 沿特征维度拼接三个投影矩阵；式 (3-37) 用一次矩阵乘法完成三份投影；式 (3-38) 再沿第二维切回三个张量。




- 合并投影把三次矩阵乘法并为一次，显著提升 GPU 利用率。
- 切分仅改变特征维度的组织方式，词元数 $n$ 保持不变。

**依据** 《大语言模型：训练、精调与对齐》第 3.2.2 节（式 (3-36)–(3-38)）

# 多头注意力的切分

$$\boldsymbol{Q} \to [\boldsymbol{Q}^{(1)}, \boldsymbol{Q}^{(2)}, \cdots, \boldsymbol{Q}^{(h)}], \quad \boldsymbol{Q}^{(i)} \in \mathbb{R}^{n \times d'} \tag{3-39}$$

$$\boldsymbol{K} \to [\boldsymbol{K}^{(1)}, \boldsymbol{K}^{(2)}, \cdots, \boldsymbol{K}^{(h)}], \quad \boldsymbol{K}^{(i)} \in \mathbb{R}^{n \times d'} \tag{3-40}$$

$$\boldsymbol{V} \to [\boldsymbol{V}^{(1)}, \boldsymbol{V}^{(2)}, \cdots, \boldsymbol{V}^{(h)}], \quad \boldsymbol{V}^{(i)} \in \mathbb{R}^{n \times d'} \tag{3-41}$$

每个头的维度为 $d' = d/h$；$\boldsymbol{Q}^{(i)}, \boldsymbol{K}^{(i)}, \boldsymbol{V}^{(i)}$ 分别表示第 $i$ 个头的查询、键和值矩阵。

**例**

$d = 512$、$h = 8$，则 $d' = 64$。

**切分方向**

沿特征维度切分，词元数 $n$ 不变。

**并行**

$h$ 个头彼此独立，可并行计算。

**依据** 《大语言模型：训练、精调与对齐》第 3.2.2 节（式 (3-39)–(3-41)）

# 各注意力头的计算

$$\boldsymbol{S}^{(i)} = \frac{\boldsymbol{Q}^{(i)}\boldsymbol{K}^{(i)^T}}{\sqrt{d'}} \quad \in \mathbb{R}^{n\times n} \tag{3-42}$$

$$\boldsymbol{S}'^{(i)} = \texttt{mask}(\boldsymbol{S}^{(i)}, \boldsymbol{M}) \tag{3-43}$$

$$\boldsymbol{P}^{(i)} = \texttt{Softmax}(\boldsymbol{S}'^{(i)}) \quad \in \mathbb{R}^{n\times n} \tag{3-44}$$

$$\boldsymbol{Z}^{(i)} = \boldsymbol{P}^{(i)}\boldsymbol{V}^{(i)} \quad \in \mathbb{R}^{n\times d'} \tag{3-45}$$

第 $i$ 个头的缩放因子为 $\sqrt{d'}$；`mask` 为逐元素掩码操作，把未来位置的分数置为 $-\infty$。

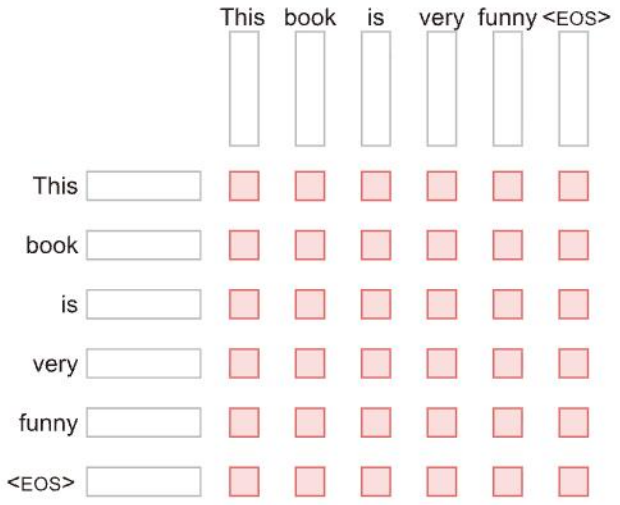



图 3.7(a)　注意力分数

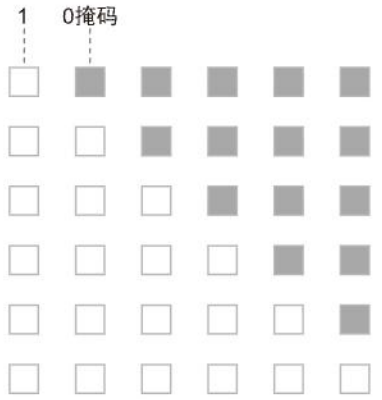



图 3.7(b)　掩码矩阵

**图源** 《大语言模型：训练、精调与对齐》图 3.7；式 (3-42)–(3-45)

# 下三角掩码

掩码矩阵 $\boldsymbol{M}$ 是一个下三角矩阵：主对角线及下三角元素为 1，上三角元素为 0（深色色块为掩码）。

$$M_{ij} = \begin{cases} 1 & \text{if } j \leqslant i \\ 0 & \text{if } j > i \end{cases}$$

掩码矩阵的定义：主对角线及其下方为 1，其余为 0。

$$M_{ij} = 0 \;\Rightarrow\; S'^{(i)}_{ij} = -\infty$$
$$\Rightarrow \exp\left(S'^{(i)}_{ij}\right) = 0$$

掩码位置的分数取 $-\infty$，其指数为 0。

$$S'^{(i)}_{ij} = \begin{cases} S^{(i)}_{ij} & \text{if } M_{ij} = 1 \\ -\infty & \text{if } M_{ij} = 0 \end{cases}$$

`mask` 按 $\boldsymbol{M}$ 中为零的位置，把对应分数置为 $-\infty$。

$$P^{(i)}_{ij} = \frac{\exp\left(S'^{(i)}_{ij}\right)}{\sum_k \exp\left(S'^{(i)}_{ik}\right)} = 0$$

分子为零，Softmax 后该位置的权重严格为 0，信息不再流入。

对第 $i$ 个位置，$\boldsymbol{Z}^{(i)}_i$ 只由位置 1 到 $i$ 的值向量加权得到。

# 拼接与输出投影

$$\boldsymbol{Z} \leftarrow [\boldsymbol{Z}^{(1)}, \boldsymbol{Z}^{(2)}, \ldots, \boldsymbol{Z}^{(h)}] \quad \in \mathbb{R}^{n \times d} \tag{3-46}$$

$$\boldsymbol{O} = \boldsymbol{Z}\boldsymbol{W}_o + \boldsymbol{b}_o \quad \in \mathbb{R}^{n \times d} \tag{3-47}$$

$\boldsymbol{W}_o \in \mathbb{R}^{d \times d}$、$\boldsymbol{b}_o \in \mathbb{R}^{d}$ 分别为输出投影层的权重矩阵和偏置。




- 拼接沿特征维度进行，把 $h$ 个 $d'$ 维的特征重新拼成 $d$ 维。
- 输出投影层融合各头信息，输出维度与块输入一致，便于残差相加与逐块堆叠。

# 前馈层的计算

前馈层通过非线性变换增强模型的特征表示能力，包含两个线性变换层及其偏置项。

$$\begin{aligned}\boldsymbol{X}_{\text{up}} &= \boldsymbol{X}\boldsymbol{W}_1 + \boldsymbol{b}_1 \\ \boldsymbol{X}_{\text{act}} &= \texttt{GELU}(\boldsymbol{X}_{\text{up}}) \\ \boldsymbol{X}_{\text{down}} &= \boldsymbol{X}_{\text{act}}\boldsymbol{W}_2 + \boldsymbol{b}_2\end{aligned}$$

$\boldsymbol{W}_1 \in \mathbb{R}^{d\times 4d}$、$\boldsymbol{b}_1 \in \mathbb{R}^{4d}$ 为升维参数；$\boldsymbol{W}_2 \in \mathbb{R}^{4d\times d}$、$\boldsymbol{b}_2 \in \mathbb{R}^{d}$ 为降维参数；$d$ 为模型隐藏层维度。

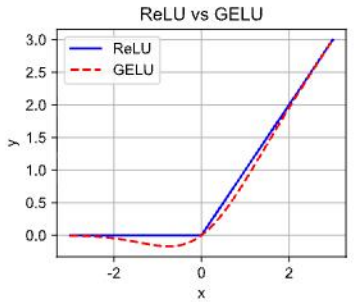
$$\boldsymbol{h}_i = \texttt{GELU}(\boldsymbol{x}_i\boldsymbol{W}_1 + \boldsymbol{b}_1)\boldsymbol{W}_2 + \boldsymbol{b}_2 \tag{3-48}$$

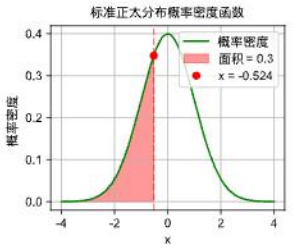
前馈层对序列中每个位置 $i$ 的词向量 $\boldsymbol{x}_i \in \mathbb{R}^d$ 独立变换，不依赖序列中的其他元素。

这种逐位置独立的结构，使前馈层能够学习不依赖于序列上下文的复杂特征模式。

**依据** 《大语言模型：训练、精调与对齐》第 3.2.2 节（式 (3-48)）

# GELU 的特性



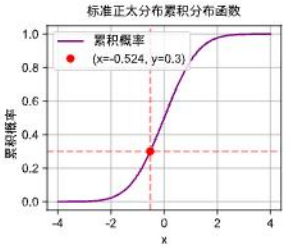




图 3.8(a)　ReLU 与 GELU　图 3.8(b)　概率密度　图 3.8(c)　累积概率

图 3.8　GELU 函数示例图

- ReLU 在 0 处不可导，会产生梯度消失等现象。
- GELU 处处可导、具有非线性，可看作 ReLU 的平滑版本，能减轻梯度消失并提高非线性表达。
- GPT-2 用 GELU 作用于升维后的特征 $\boldsymbol{X}_{\text{up}}$。

**图源** 《大语言模型：训练、精调与对齐》图 3.8

# GELU 与误差函数的定义

$$\mathbf{GELU}(x) = x \cdot \Phi(x) \tag{3-49}$$

$\Phi(x)$ 是标准正态分布的累积分布函数（CDF）：$\Phi(x) = \frac{1}{2}\left[1 + \mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right]$，erf 是误差函数。

$$\mathrm{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \tag{3-50}$$

误差函数 $\mathrm{erf}(x)$ 是高斯（正态）分布概率密度函数（图 3.8(b)）的积分，其曲线见图 3.8(c)。

- GELU 用输入 $x$ 乘以它落在标准正态分布左侧的概率 $\Phi(x)$：负输入被平滑地压制，正输入近似线性通过。
- 与 ReLU 的硬截断相比，GELU 在 0 附近平滑过渡，处处可导。

# CDF 的推导*

式 (3-49) 把 $\Phi(x)$ 写成 $\frac{1}{2}[1+\mathrm{erf}(x/\sqrt{2})]$。这一步不是定义，而是由两个归一化常数与一次变量替换推出来的。

$$\int_{-\infty}^{+\infty} e^{-t^2}\,dt=\sqrt{\pi},\qquad \int_{-\infty}^{+\infty} e^{-t^2/2}\,dt=\sqrt{2\pi}$$

高斯积分：两者的概率密度分别在除以 $\sqrt{\pi}$、$\sqrt{2\pi}$ 后归一。

$$\mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)=\frac{2}{\sqrt{\pi}}\int_0^{x/\sqrt{2}} e^{-t^2}\,dt=\sqrt{\frac{2}{\pi}}\int_0^{x} e^{-u^2/2}\,du$$

令 $t=u/\sqrt{2}$，则 $dt=du/\sqrt{2}$，积分限 $0\to x/\sqrt{2}$ 变为 $0\to x$。

$$\Phi(x)=\frac{1}{\sqrt{2\pi}}\int_{-\infty}^{x} e^{-t^2/2}dt=\frac{1}{2}+\frac{1}{\sqrt{2\pi}}\int_0^{x} e^{-t^2/2}dt$$

正态密度关于 0 对称，左半轴积分为 $\frac{1}{2}$。

$$\frac{1}{\sqrt{2\pi}}\int_0^{x} e^{-t^2/2}dt=\frac{1}{2}\,\mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)$$

$$\Longrightarrow\quad \Phi(x)=\frac{1}{2}\left[1+\mathrm{erf}\left(\frac{x}{\sqrt{2}}\right)\right]$$

代入上一行，即得式 (3-49) 中的 $\Phi(x)$ 表达式。

- 由此还可得三条常用性质：$\Phi(0)=\frac{1}{2}$，$\Phi(-x)=1-\Phi(x)$，$x\to\pm\infty$ 时 $\Phi(x)\to 1$ 或 $0$。
- 第三条正是 GELU 的两端行为：正输入近似恒等、负输入被压到 $0$，且中间没有折点。

**依据** 《大语言模型：训练、精调与对齐》第 3.2.2 节（式 (3-49)、(3-50)）

# GELU 的近似算法

$\Phi(x)$ 需要积分或查表，逐元素计算的代价高。工程实现把门函数换成初等函数，用 $x\,\sigma(\alpha x)$ 的曲线去逼近 $x\,\Phi(x)$。

$$\mathrm{GELU}(x) \approx x\,\sigma(\alpha x), \quad \sigma(z) = \frac{1}{1+e^{-z}}$$

把 $\Phi(x)$ 换成 logistic CDF，只需一次指数运算；$\alpha$ 决定过渡的陡峭程度。

$$\mathrm{GELU}(x) \approx \frac{x}{2}\left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right)\right]$$

tanh 近似：GPT-2 采用的实现；三次项修正 $x$ 较大时的偏差。

- $\alpha = 10$ 时负半轴几乎直接压到 0，最接近 ReLU；$\alpha = 1$ 过渡偏软，负半轴压制不足。
- $\alpha = 1.702$ 使近似曲线与精确 GELU 几乎重合，是"一次指数换一个 CDF"的最优折中。
- $\alpha = 1$ 的这一族函数即 Swish（$\beta$ 可学习时记作 $x\,\sigma(\beta x)$，$\beta = 1$ 时称 SiLU）。

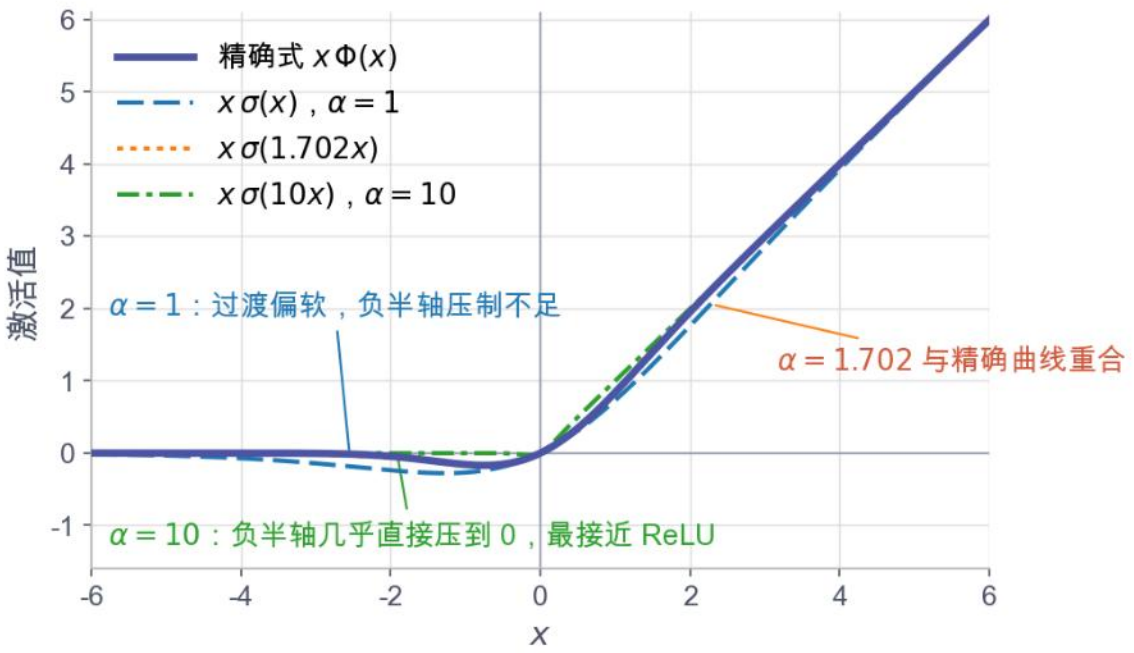



图 3.8b　$x\,\sigma(\alpha x)$ 对 GELU 的逼近：$\alpha = 1$、$1.702$、$10$

图 《大语言模型：训练、精调与对齐》图 3.8；式 (3-49)

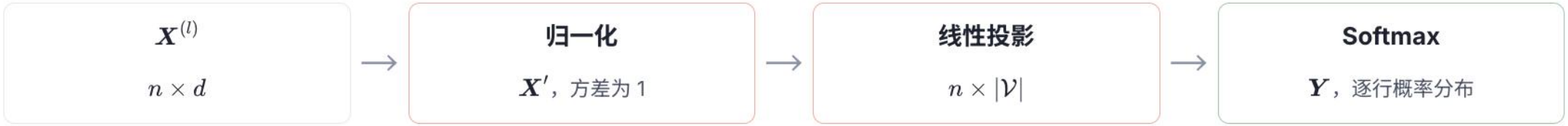
# 输出层

解码块输出 $\boldsymbol{X}^{(l)} \in \mathbb{R}^{n\times d}$。因为采用前置归一化，需在解码器后增加归一化层，把输出方差变换为 1。

$$\boldsymbol{X}' = \texttt{LayerNorm}(\boldsymbol{X}^{(l)}) \tag{3-51}$$

$$\boldsymbol{Y} = \texttt{Softmax}(\boldsymbol{X}'\boldsymbol{W}_{\text{out}} + \boldsymbol{b}_{\text{out}}) \tag{3-52}$$

$\boldsymbol{W}_{\text{out}} \in \mathbb{R}^{d\times|\mathcal{V}|}$ 为线性投影矩阵，$\boldsymbol{b}_{\text{out}}$ 为偏置；$\boldsymbol{Y} \in \mathbb{R}^{n\times|\mathcal{V}|}$ 的每一行对应一个位置的下一个词元预测 $p_\theta(\cdot|\boldsymbol{x}_{<i})$。




- $\boldsymbol{X}'$ 为 $\boldsymbol{X}^{(l)}$ 的层归一化输出；$\boldsymbol{Y}$ 表示 $n$ 个概率分布，每个对应序列位置 $i$ 的下一个词元预测。
- $|\mathcal{V}|$ 为词表大小，GPT-2 中为 50 257。

**依据** 《大语言模型：训练、精调与对齐》第 3.2.3 节（式 (3-51)、(3-52)）

# 训练损失

用交叉熵损失衡量预测分布与真实标签之间的误差，一个序列的损失函数定义为：

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\log p_\theta(x_i|\boldsymbol{x}_{<i}) \tag{3-53}$$

$p_\theta(x_i|\boldsymbol{x}_{<i})$ 是模型预测的第 $i-1$ 个位置的下一个词元 $x_i$ 的概率。

## 与式 (3-1) 的差别

式 (3-1) 对整段序列求和，式 (3-53) 再除以 $n$，得到每个位置的平均损失。

平均化使不同长度的序列之间损失可比。

## 并行预测

得益于 Transformer 的并行预测特性，GPT-2 前向计算中可并行地对所有 $n$ 个词元做下一个词元预测。

训练一次前向即得到 $n$ 个监督信号。

# 3.3

# BPE 编码

字节对编码结合字符级与词级分词的优点：用迭代合并高频字节对的方式构建子词词表，兼顾覆盖率与序列长度。

本节内容

**3.3.1 BPE 编码算法原理**

3.3.2 BPE 词表训练

3.3.3 BPE 编码

3.3.4 BPE 解码

# 子词分词的作用

一种基础的分词方法是字符级编码：中文里每个汉字通常视为一个独立字符；英文里空格也算一个字符。

**字符级**

“hello world”编码为 ‘h’, ‘e’, ‘l’, ‘l’, ‘o’, ‘_’, ‘w’, ‘o’, ‘r’, ‘l’, ‘d’（‘_’表示空格）。

能处理任意文本，但编码后的序列过长：共 11 个词元。

**词级**

同一句子可能只需 3 个词元：‘hello’, ‘_’, ‘world’。

序列短，但词表要覆盖所有词形，遇到未登录词难以处理。

BPE 是介于二者之间的子词（subword）分词方法，结合了字符级与词级分词的优点，通过把单词拆分为更小的、可重用的子词来工作。

- 例如训练好的 BPE 分词器可能把 “happiness” 拆分为 ‘happi’, ‘ness’。
- 它能共享子词单元来表示不同形态的词汇（“happy”“happily”“unhappy”），特别适合英语等屈折变化丰富的语言。
- 中文同样适用：根据语料频率，“计算机”可能拆为 ‘计’,‘算’,‘机’ 或 ‘计算’,‘机’。
- GPT-2 在 WebText 语料库上训练分词器，最终词表包含 $|\mathcal{V}| = 50\,257$ 个词元。

# BPE 算法

BPE 算法最初用于数据压缩。应用于自然语言处理构建分词词表时，其核心是迭代地合并语料中最频繁共现的字节对，生成新的子词。

**字符单元序列**
语料切分为最小字符

→

**统计字节对**
相邻对的频次 $C(x, y)$

→

**合并最高频对**
加入词表 $\mathcal{V}$

→

**重复**
直到预设词表大小

- 经过多轮合并后，高频出现的字符序列被合并成新的词元加入词表。
- 基于最终词表，可以把文本编码为词元编号序列，也可以把编号序列解码回原始文本。
- BPE 的编码和解码都具有唯一性：确定的词表与合并列表下，结果唯一。

# BPE 第 1 步　初始化词表

- 创建一个空的初始词表 $\mathcal{V}$。
- 把输入语料分割成最小的字符单元序列。
- 把语料中出现的所有**唯一字符**加入 $\mathcal{V}$，作为初始词表。
- 示例文本 “abcabcab” 切分为（‘a’,‘b’,‘c’,‘a’,‘b’,‘c’,‘a’,‘b’），只有 3 个唯一字符 ‘a’、‘b’、‘c’。

> **要点**　此时词表只有 3 项，编号 1、2、3 分别对应 a、b、c；后续合并得到的新词元从编号 4 开始追加。

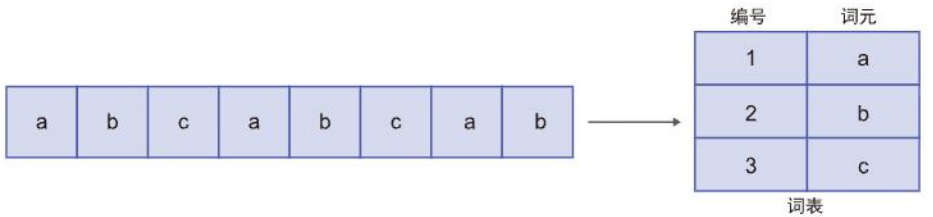



图 3.9　BPE 词表初始化

**图源** 《大语言模型：训练、精调与对齐》图 3.9

# BPE 第 2–3 步　统计频次与选取

字符合邻组成一对 $(x, y)$，$C(x, y)$ 为该相邻对在序列中出现的频次；在其中选取频次最高者合并：$(x^*, y^*) = \arg\max_{(x,y)} C(x, y)$。

序列 ‘a’,‘b’,‘c’,‘a’,‘b’,‘c’,‘a’,‘b’ 的全部相邻对：

| 相邻对 | 出现位置 | 频次 $C$ | 是否选中 |
|---|---|---|---|
| (‘a’,‘b’) | 第 1、4、7 对 | 3 | **是（最大）** |
| (‘b’,‘c’) | 第 2、5 对 | 2 | 否 |
| (‘c’,‘a’) | 第 3、6 对 | 2 | 否 |

- $C(\text{‘a’},\text{‘b’}) = 3$ 最大，选择 (‘a’,‘b’) 作为本轮合并对象。
- 若出现并列最高频，则需要一条确定的裁决规则，以保证训练结果可复现。

# BPE 第 4 步　更新词表与序列

- 新词元 $c_{\text{new}} = x^* + y^*$，这里 $+$ 表示字符串拼接；本例 $c_{\text{new}} =$ ‘ab’。
- 把 $c_{\text{new}}$ 添加至词表，并记录本次合并三元组 (‘a’,‘b’,‘ab’) 到合并列表 $M$。
- 用 $c_{\text{new}}$ 更新序列中所有出现 $(x^*, y^*)$ 的位置：‘ab’,‘c’,‘ab’,‘c’,‘ab’。
- 图 3.10 中可见编号 4 的词元 ‘ab’ 已加入词表。

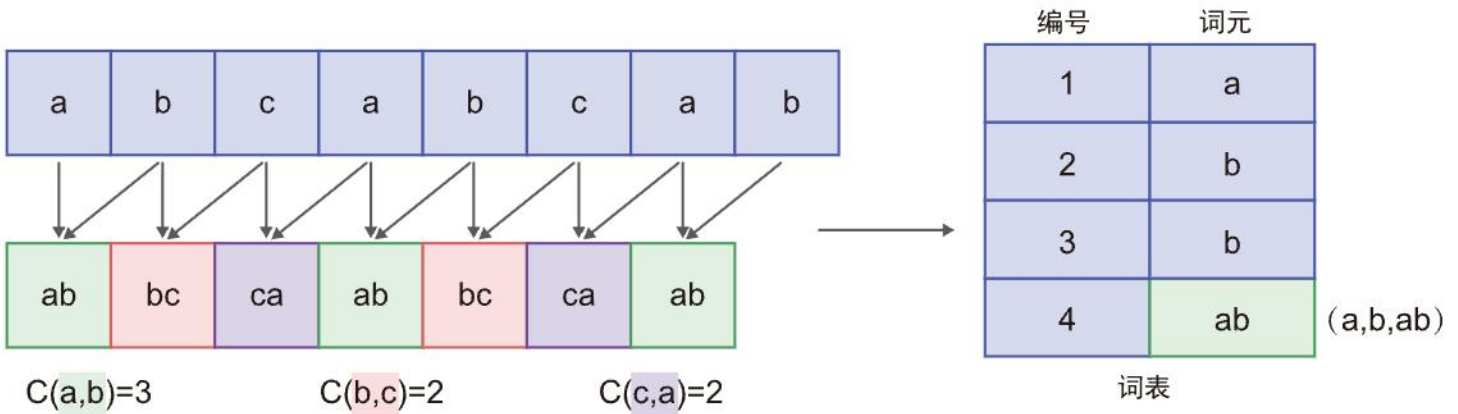



图 3.10　BPE 训练过程图（首轮合并）

合并列表 $M$ 记录了“谁由谁合并而来”，它是后续编码时决定合并顺序的依据。

**图源** 《大语言模型：训练、精调与对齐》图 3.10

# BPE 第 5 步　迭代至词表上限

- 基于更新后的序列 ‘ab’,‘c’,‘ab’,‘c’,‘ab’，返回第 2～5 步，重复统计、选择合并、更新词表与序列。
- 第二轮：$C(\text{‘ab’},\text{‘c’}) = 2$、$C(\text{‘c’},\text{‘ab’}) = 2$，两者并列最高。
- 合并得到 (‘ab’,‘c’,‘abc’) 与新词元 ‘abc’，编号 5。
- 序列更新为 ‘abc’,‘abc’,‘ab’。

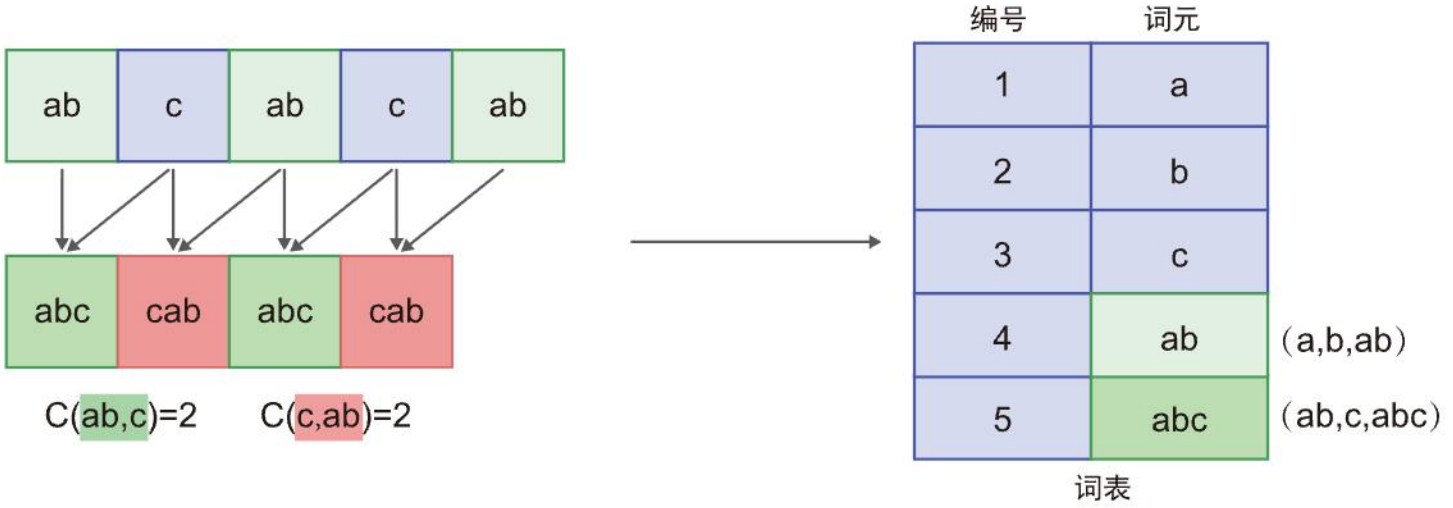



图 3.11　BPE 训练过程图（第二轮合并）

训练在词表达预设上限时停止。本例最终词表为 $\{\text{a:1}, \text{b:2}, \text{c:3}, \text{ab:4}, \text{abc:5}\}$，合并表 $M = \{(\text{‘}a\text{’}, \text{‘}b\text{’}, \text{‘}ab\text{’}), (\text{‘}ab\text{’}, \text{‘}c\text{’}, \text{‘}abc\text{’})\}$。

**图源** 《大语言模型：训练、精调与对齐》图 3.11

# BPE 的训练结果

训练结束后保存词表 $\mathcal{V}$ 与合并列表 $M$。原文本 “abcabcab” 最终编码为 ‘abc’,‘abc’,‘ab’。

**词表 $\mathcal{V}$**

a:1　b:2
c:3
ab:4
abc:5

**合并表 $M$**

(‘a’,‘b’,‘ab’)
(‘ab’,‘c’,‘abc’)

8 个字符被压缩为 3 个词元；合并表决定了编码时“先合并谁、后合并谁”。

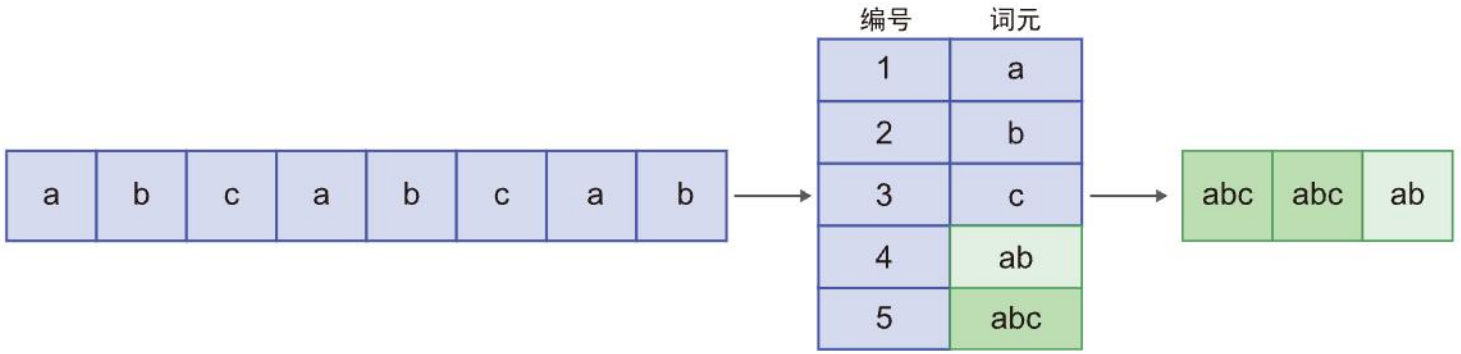



图 3.12　BPE 最终构建词表和编码结果

**图源** 《大语言模型：训练、精调与对齐》图 3.12

# BPE 编码　第一轮合并

- 待编码文本 “babcabc” 先分割为 ‘b’,‘a’,‘b’,‘c’,‘a’,‘b’,‘c’。
- 构建相邻对 (‘b’,‘a’)、(‘a’,‘b’)、(‘b’,‘c’)、(‘c’,‘a’)、(‘a’,‘b’)。
- 查合并表 $M$，只有 (‘a’,‘b’) 可以合并。
- 更新序列为 ‘b’,‘ab’,‘c’,‘ab’,‘c’。

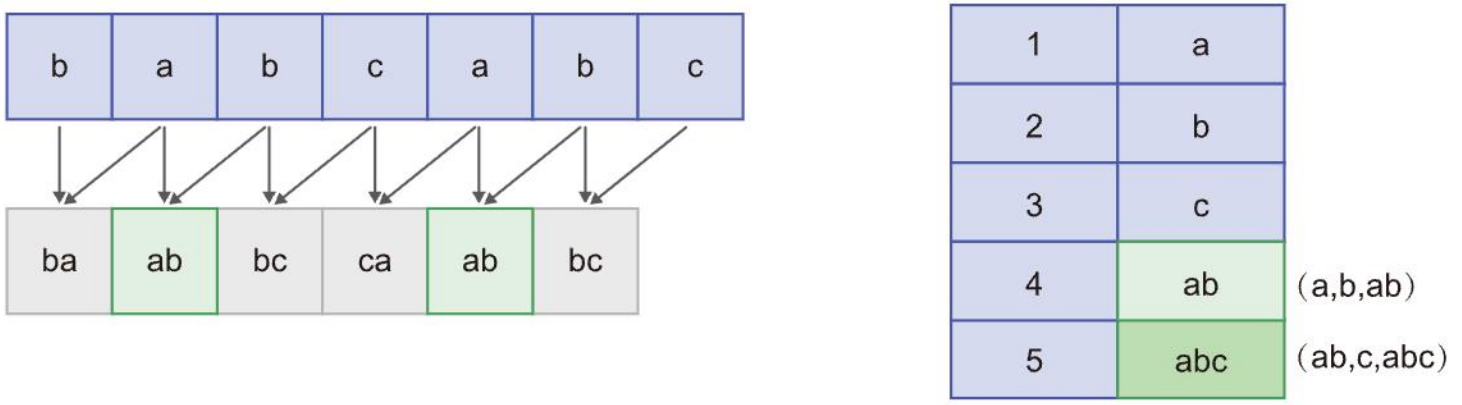



图 3.13　BPE 编码首轮

> **注意**　即使词表里有 ‘abc’，原词元序列也是先把 ‘a’ 与 ‘b’ 合并成 ‘ab’，再与 ‘c’ 合并成 ‘abc’——合并必须按合并表的顺序进行，不能一步跳到底。

**图源**　《大语言模型：训练、精调与对齐》图 3.13

# BPE 编码　第二轮合并

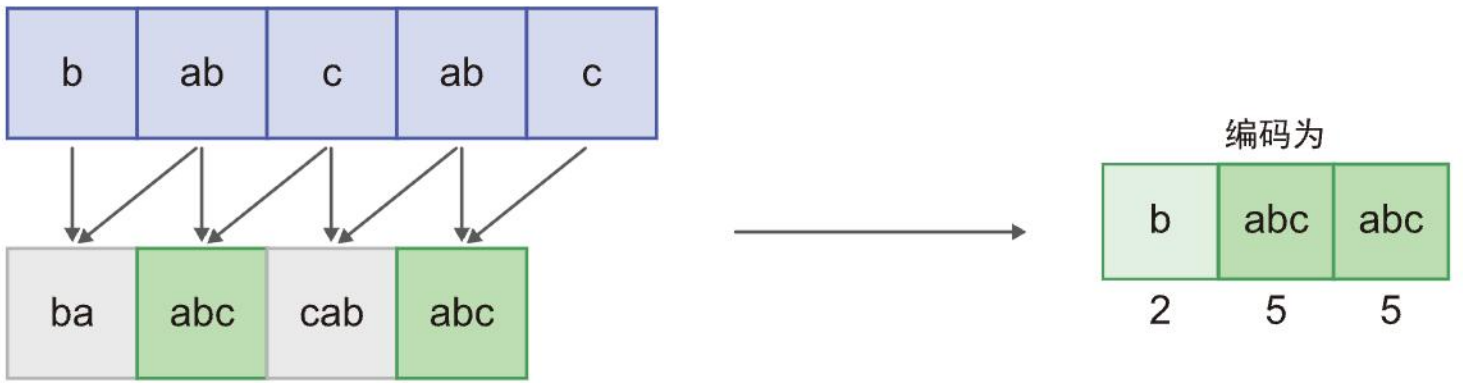



图 3.14　BPE 编码第二轮

- 按更新后的序列 ‘b’,‘ab’,‘c’,‘ab’,‘c’ 重新构建相邻对，继续查询合并表。
- 相邻对 (‘ab’,‘c’) 在 $M$ 中，合并为 ‘abc’。
- 最终得到 ‘b’,‘abc’,‘abc’。
- 按词表序号编码为词元编号序列 ‘2’,‘5’,‘5’。

**复杂度**　假设词元序列为 ‘ab’,‘c’,‘a’,‘b’，此时 (‘ab’,‘c’) 与 (‘a’,‘b’) 都可合并；优先合并 ‘ab’，因为它在词表中编号更小，说明其合并频次更大。

从词表训练和编码可以看出，BPE 的词表基于统计和规则构建，并非按语义划分文本；在确定的词表与合并列表下，分词结果唯一。

**图源** 《大语言模型：训练、精调与对齐》图 3.14

# BPE 解码

BPE 的解码从词元编号序列的每个编号出发，在词表里找到对应词元，再把所有词元文本拼接起来。

给定编号序列 '1','2','1','4','5'：

| 位置 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 词元编号 | 1 | 2 | 1 | 4 | 5 |
| 查表得到 | a | b | a | ab | abc |

$$\text{'a'} + \text{'b'} + \text{'a'} + \text{'ab'} + \text{'abc'} = \text{"abaababc"}$$

$+$ 表示字符串拼接；解码结果即原文本 "abaababc"。

- 解码只需词表，不需要合并表 $M$；合并表只在编码（分词）时使用。
- 因为编码只是把字符序列改写为合并后的词元，所以编码与解码互为逆过程，可无损还原原文。

# 讨论：<<eos> 如何切分?

词表中 '<''<' 已合成 '<<'，而 <eos> 排在后面。"3[]5谁大?<<eos>" 的结尾该切成哪种?

### (A) '<<' + 'eos' + '>'

按合并表贪心：'<<' 先合并占掉开头两个 '<'，剩下的 'eos' 与 '>' 都无法再合并。

3 个词元。纯 BPE 算法的结果。

### (B) '<' + '<eos>'

特殊词元优先：先把 <eos> 整段识别出来，它前面的 '<' 单独成词元。

2 个词元。多数生产分词器的结果。

- **纯 BPE 下是 (A)**：合并表按序号执行，'<<' 先于任何字符配对环境且不回退。
- **实践中是 (B)**：<eos> 属**特殊词元**，不参与 BPE；分词器先切它、再对每段做 BPE。
- **结论**：答案取决于分词器的规则顺序，而非"哪个更短"；同一文本在不同实现里切法不同，但各自确定。

> **延伸** 没有边界保护时，"<eos>" 可能被切成 '<'+'eos'+'>'，终止语义被削弱——这正是特殊词元要在训练前从词表中隔离的原因。但也不必说成"彻底失去"：模型见过大量由 '<'、字母与 '>' 连绵组成的文本，一定程度上仍能从连续词元的组合中认出这是标签式片段；代价只是**不再可靠**——从"认一个确定的词元"退化为"靠上下文去猜"。

# 3.4

# GPT 模型生成

本节内容

**3.4.1 GPT 模型推理**

3.4.2 GPT 生成方法

3.4.3 本节小结

预训练后的 GPT 逐步生成词元：新词元追加为输入的一部分，直到遇到 <EOS> 或达到最大长度。

# 自回归推理

GPT 的生成过程也称模型推理（Inference）：根据输入序列 $\boldsymbol{x}_{<t} = (x_0, x_1, \cdots, x_{t-1})$，模型输出下一个词元的概率分布 $p_\theta(\cdot|\boldsymbol{x}_{<t})$。

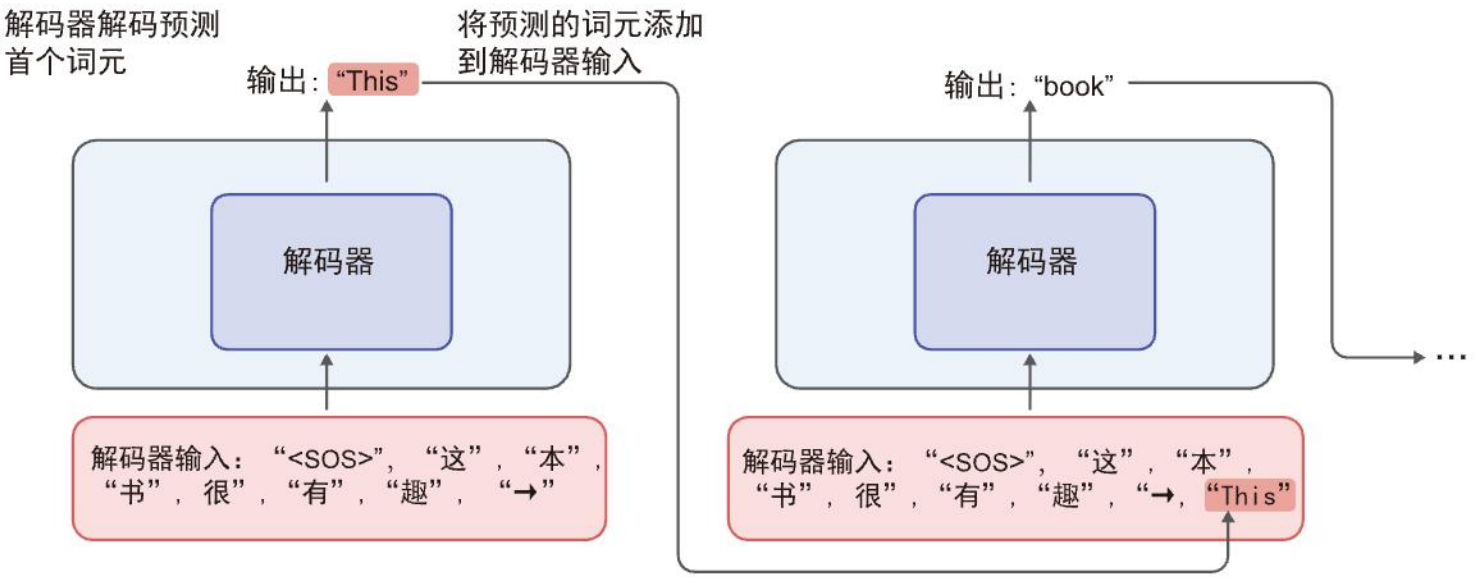



图 3.15　GPT 推理阶段的串行词元预测

$$x_t = \arg\max_{v \in \mathcal{V}} p_\theta(\cdot \mid \boldsymbol{x}_{<t})$$

式 (3-54)　贪婪搜索（Greedy Searching）：选择概率最高的词元作为输出。

- 每生成一个词元就追加到输入，再预测下一个；图中第二轮输入已含 “This”。
- 推理必须逐词元串行推进，与训练时的并行预测相反。
- 生成终止符 <EOS> 或达到最大长度时终止。

**图源**　《大语言模型：训练、精调与对齐》图 3.15；式 (3-54)

# 贪婪解码的伪代码

```
class DecodingModel(nn.Module):
    def __init__(self, dim, heads, vocab_size):
        super().__init__()
        self.embd = nn.Embedding(vocab_size, dim)
        self.decoder = MultiHeadAttention(dim, heads)
        self.norm = nn.LayerNorm(dim)
        self.lm_heads = nn.Linear(dim, vocab_size)

    def forward(self, x, mask):
        X = self.embd(x)
        X = self.decoder(X, mask)
        H = self.norm(X)
        logits = self.lm_heads(H)
        return logits

vocab_size = 100
heads = 8
dim = 64
max_new_tokens = 10
EOS_token_ID = 0
```

```
# 1 个输入，序列长度为 16
bs = 1
seq_len = 16
x = torch.randint(low=1, high=vocab_size,
                  size=(bs, seq_len))
mask = torch.triu(torch.ones(1, 1,
                             seq_len+max_new_tokens,
                             seq_len+max_new_tokens,
                             dtype=torch.bool))[:, :, 1:, :-1]
model = DecodingModel(dim, heads, vocab_size)

for i in range(max_new_tokens):
    logits = model(x, mask[:, :, :seq_len+i, :seq_len+i])
    next_tokens = torch.argmax(logits, dim=-1)
    x = torch.cat((x, next_tokens[:, [-1]]), dim=-1)
    if next_tokens[0, -1] == EOS_token_ID:
        break
```

每轮取最后一个位置的 $\arg\max$ 并追加到输入，直到生成 <EOS> 或达到 `max_new_tokens`。

**依据** 《大语言模型：训练、精调与对齐》第 3.4.1 节（贪婪解码伪代码）

# 采样解码的必要性

贪婪搜索对相同输入提示会生成确定性的文本序列；但概率语言模型的本质是输出**下一个词元的概率**，天然具有随机性。

## 贪婪搜索

每次抓球都必然选当前概率最高的球（红球），十次结果只能是“红红红红红红红红红红”。

输出确定，但会丢失低概率的合理候选。

## 按概率采样

箱中 10 球：7 红、2 白、1 黑，概率分别为 0.7、0.2、0.1。十次有放回抓取可能是“红红红白红红红黑红黑”。

概率最高者更常被选中，但低概率者也有机会。

- 采样（Sampling）即从模型给出的概率分布中随机选择下一个词元。
- 由此引入多种生成方法：采样、温度采样、Top-$K$、Top-$P$ 与束搜索，分别权衡多样性与准确性。

**依据** 《大语言模型：训练、精调与对齐》第 3.4.2 节

# 随机采样解码

采样是一种引入随机性以生成多样化输出的解码策略：模型输出下一个词元的概率分布，基于该分布采样即可随机选择。

- 可类比为有放回的抓球实验：每个候选词元对应一个球，被选中的概率由模型预测给出。
- 例如输入“计算机科”，预测分布可能为 ‘学’:0.6、‘技’:0.2、‘研’:0.1。
- 词元 ‘技’ 概率为 0.2，虽非最高，但仍有机会被选中。

> **取舍** 采样显著提高文本生成的多样性，但也可能产生不合理或错误的输出，更适用于小说创作、标题生成、活动策划等需要创造力的任务。

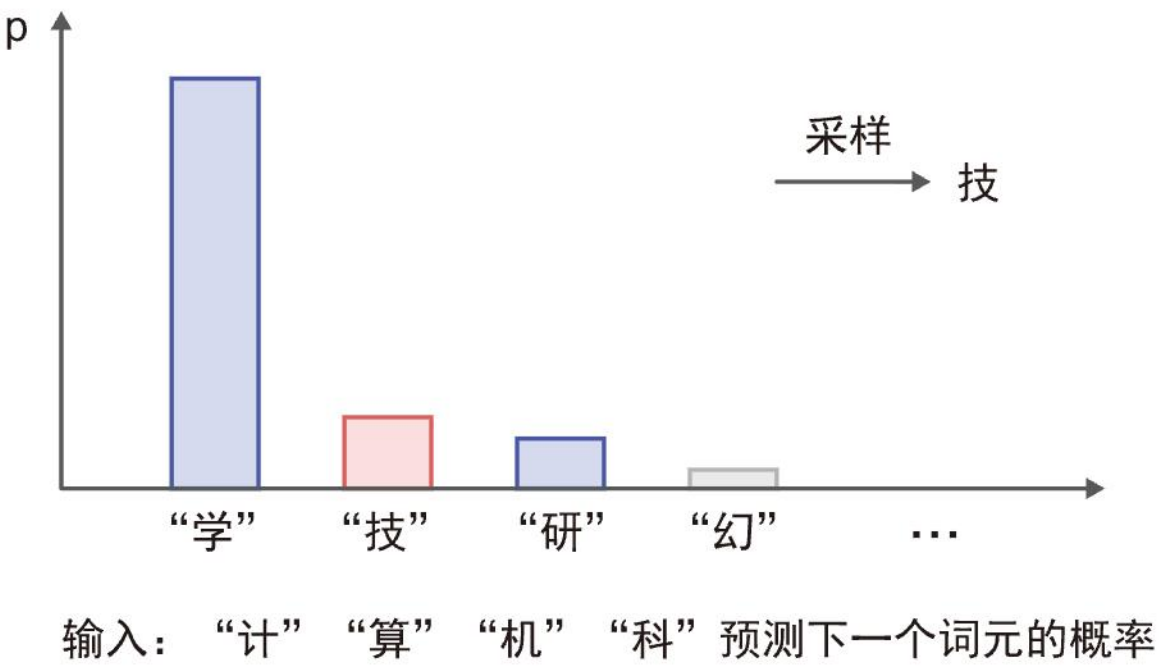



图 3.16　采样解码示例

**图源** 《大语言模型：训练、精调与对齐》图 3.16

# 温度采样

模型最后一层的输出称为 logits（原始分数），需经 Softmax 转为概率分布。温度采样在 Softmax 之前把 logits 除以温度 $t$。

$$\mathtt{Softmax}(\boldsymbol{x}_i, t) = \frac{\mathrm{e}^{\boldsymbol{x}_i/t}}{\sum_{j=0}^{|\mathcal{V}|} \mathrm{e}^{x_{ij}/t}} \tag{3-55}$$

$\boldsymbol{x}_i \in \mathbb{R}^{|\mathcal{V}|}$ 标记 logits 的一个行分数向量。

- **低温** $(0 < t < 1)$：分布更集中（陡峭），高概率词元的优势被放大，低概率词元被抑制；生成更确定、更保守，适用于解题、代码生成与逻辑推理。
- **高温** $(t > 1)$：分布更分散（平缓），低概率词元机会增加；生成更多样、更具探索性，适用于创意写作。
- $t \to \infty$：分布趋近均匀，所有候选词元的概率几乎相等，随机性最大。

# 不同温度下的输出分布

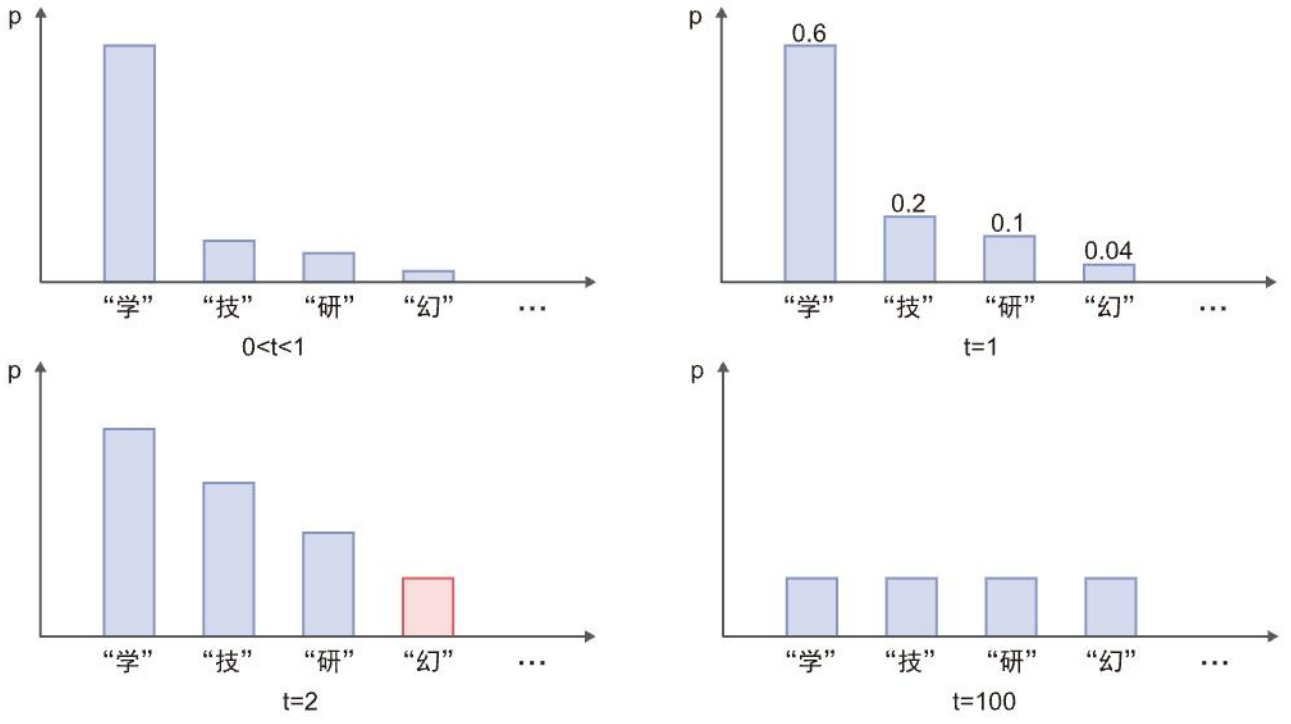



图 3.17　基于温度改变概率分布

- $0 < t < 1$：概率集中在少数高概率词元上，曲线很陡。
- $t = 1$：原始分布，例如 0.6、0.2、0.1、0.04。
- $t = 2$：曲线变平缓，词元“幻”的概率相对增加，更可能被采样到。
- $t = 100$：接近均匀分布，几乎完全随机。

温度是一个“先验强弱”的旋钮：调低即更相信模型的最高分，调高即更愿意探索次优候选。

**图源** 《大语言模型：训练、精调与对齐》图 3.17

# Top-$K$ 采样

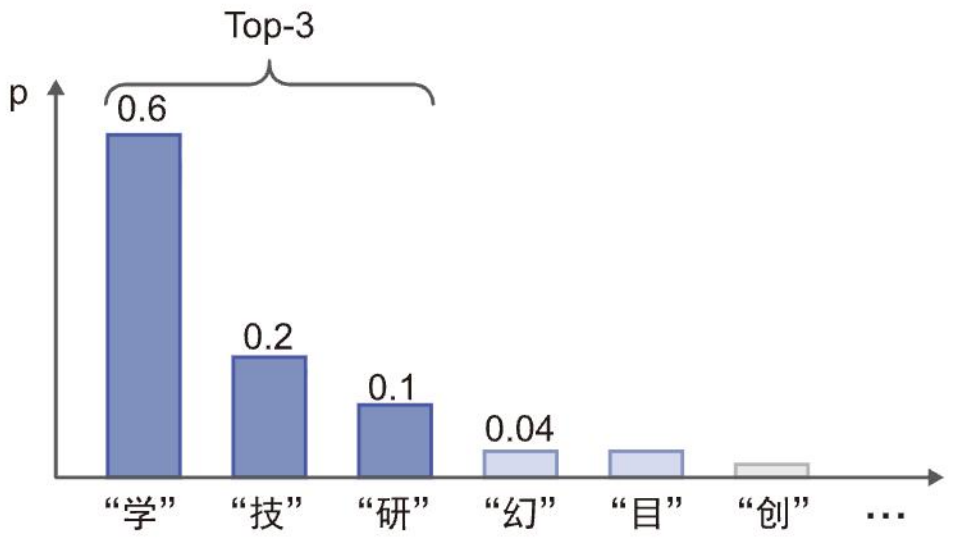



图 3.18(a)　预测分布 1

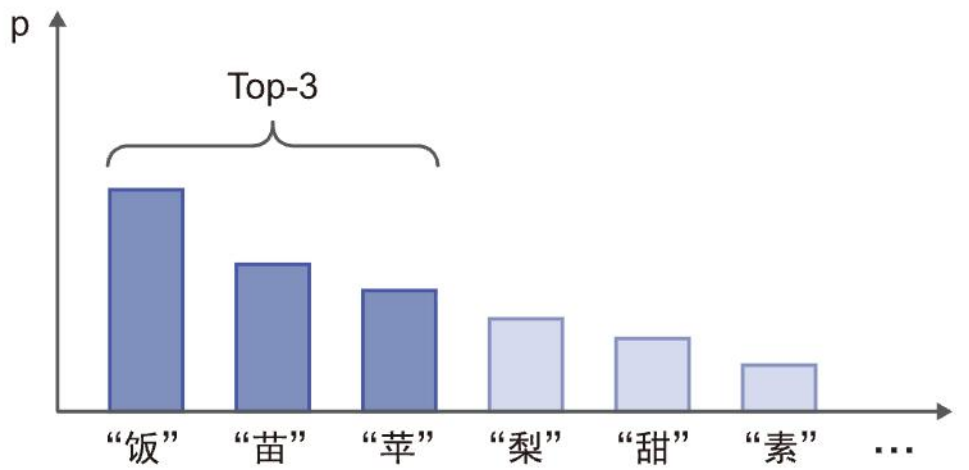



图 3.18(b)　预测分布 2

图 3.18　Top-$K$ 采样

- 原理：在下一个词元的概率分布中，选取概率最大的 $K$ 个词元构成候选集合，仅在该集合内采样；图 (a) 在 Top-3 词元上采样。
- 实现：把非 Top-$K$ 词元的 logits 置为 $-\infty$，再做 Softmax 归一化，其概率趋近 0。
- 局限：不同位置的分布差异可能很大，固定 $K$ 会过度截断，如图 (b) 把本应具有合理概率的词元排除在外。

**图源** 《大语言模型：训练、精调与对齐》图 3.18

# Top-$P$ 采样

- 为解决固定 $K$ 难以自适应的问题，Top-$P$ 采样（又称核采样，Nucleus Sampling）被提出。
- 先把词元按概率降序排列，选择累积概率刚好超过阈值 $p$ 的**最小**词元集合，仅在该集合内采样。
- 阈值 $p \in (0, 1]$，候选数量随当前分布动态调整。
- 图 3.19(b) 中，Top-3 会把"梨""甜"等高概率词元错误截断，Top-$P$ 则能保证包含最可能的词元。

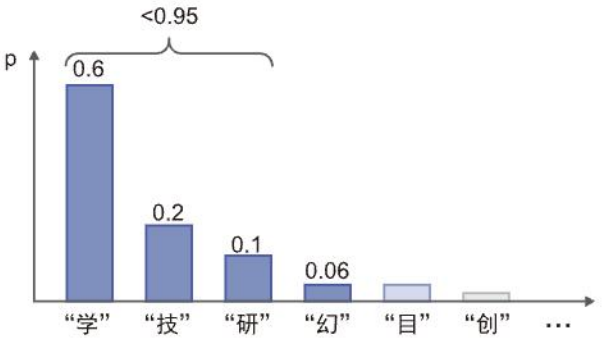



图 3.19(a)　预测分布 1

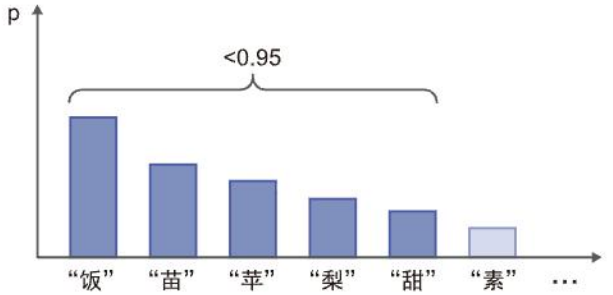



图 3.19(b)　预测分布 2

**组合**　Top-$K$ 与 Top-$P$ 的先后取决于采样策略：从效率看，先 Top-$K$ 缩小候选集、再 Top-$P$ 累加筛选更省算力。但无论顺序如何，一旦干预原始输出分布，都必然带来对原始分布的偏离——截断与重归一化会改变词元间的相对概率，因此需要在多样性与保真度之间权衡。

图 3.19　Top-$P$ 采样（示例输入"今天吃"）

**图源**　《大语言模型：训练、精调与对齐》图 3.19

# 束搜索

序列生成的最优解是全局联合概率最大的序列，但搜索空间高达 $|\mathcal{V}|^L$，贪婪搜索每步只看当前最高概率，极易陷入局部最优。

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = p(y_1, y_2, \cdots, y_L \mid \boldsymbol{x}) = \prod_{i=1}^{n} P(y_i \mid \boldsymbol{x}, y_1, y_2, \cdots, y_{i-1}) \tag{3-56}$$

$$\arg \max_{y_1, y_2, \cdots, y_t} \prod_{i=1}^{t} p(y_i \mid \boldsymbol{x}, y_1, y_2, \cdots, y_{i-1}) \tag{3-57}$$

$$\arg \max_{y_1, y_2, \cdots, y_n} \sum_{i=1}^{t} \log p(y_i \mid \boldsymbol{x}, y_1, y_2, \cdots, y_{i-1}) \tag{3-58}$$

式 (3-56) 把联合概率分解为条件概率的乘积；式 (3-57) 定义最优序列；式 (3-58) 取对数，既避免数值下溢，又把乘法变为加法。

- 束搜索每一步只保留累积概率最高的 $B$ 个候选序列（$B$ 为束宽），并基于此集合继续扩展。
- 在计算开销可控的前提下，找到质量优于贪婪搜索的次优解。

# 束搜索示例

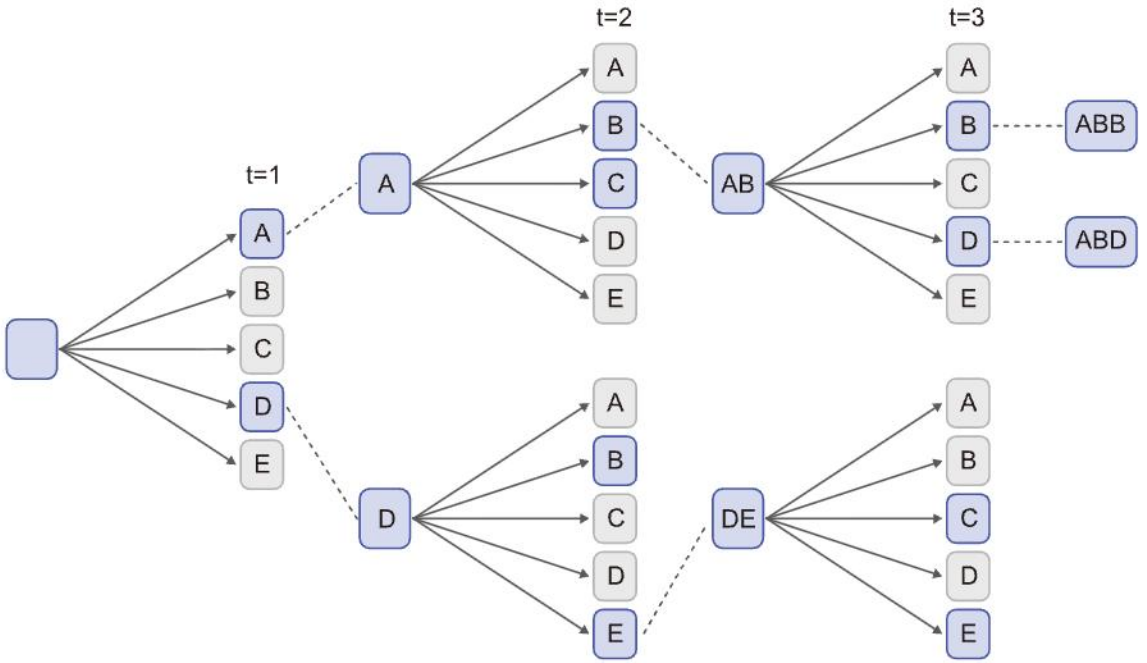



图 3.20　束搜索解码流程示例

限制每一步扩展的节点数量，是防止搜索爆炸的关键；束宽 $b = 1$ 时退化为贪婪搜索。

- $t = 1$：生成 5 个词元，选择概率最高的 $b = 2$ 个（图中 'A' 和 'D'）作为初始候选序列。
- $t = 2$：扩展 'A' 得 'A-B'、'A-C'；扩展 'D' 得 'D-B'、'D-E'。
- 计算四条路径的累积对数概率，如 A-B 为 $\log p(\text{'A'}|\boldsymbol{x}) + \log p(\text{'B'}|\boldsymbol{x}, \text{'A'})$，保留得分最高的 2 条（假设 A-B 与 D-E）。
- **后续**：重复"扩展 $b$ 条、打分、保留前 $b$ 条"，直到所有候选都遇到 <EOS> 或达到最大长度。

**图源** 《大语言模型：训练、精调与对齐》图 3.20

# 长度惩罚

当有路径提前拓展到 <EOS> 时，其余节点仍继续扩展 $b$ 个。不同长度的候选路径在最终判别时，可在路径得分上乘以长度惩罚因子 $L^{\alpha}$ 的倒数。

$$\frac{1}{L^{\alpha}} \log p(\boldsymbol{y} \mid \boldsymbol{x}) = \frac{1}{L^{\alpha}} \sum_{t=1}^{L} \log p(y'_t \mid y_1, y_2, \cdots, y_{t-1}, \boldsymbol{x}) \tag{3-59}$$

$L$ 为生成文本长度，$\boldsymbol{x}$ 为输入的词元序列，$y_t$ 为逐个生成的词元；通常 $\alpha = 0.75$ 或 $\alpha = 0$。

- 对数概率随长度单调下降，若直接比较，短序列天然占优。
- 除以 $L^{\alpha}$ 做长度归一化：$\alpha = 0$ 即不惩罚，$\alpha = 0.75$ 为常用的折中。
- 此外还有重复惩罚、典型性采样等技术，可与上述方法组合使用。

# 生成方法的分类

在 GPT 的生成中，多种策略可以组合使用。除贪婪解码、束搜索外，还有温度采样、Top-$K$、Top-$P$、重复惩罚、长度惩罚和典型性采样等；Transformers 库实现了这些方法。

## 启发式搜索算法

如束搜索：需要定义评估指标（如概率分数），据此判断是否进一步扩展搜索路径。

在候选序列空间中搜索，代价是多次前向计算。

## 修改输出分布

如温度采样、Top-$K$、Top-$P$ 与典型性采样：直接改写下一个词元的概率分布，再从中采样。

每步只需一次前向，靠分布形状控制多样性与可控性。

> **选择** 要确定、可复现的结果用贪婪搜索或低温度；要多样性用采样与高温；要兼顾质量用 Top-$K$、Top-$P$；要逼近全局最优用束搜索。

# 3.5 小结

本节内容

**预训练范式的意义**

学习与搜索的双重可扩展性

对外的辐射与后续架构

GPT 把任务学习转变为预训练学习：纯解码器架构 + 下一个词元预测，带来强大的扩展能力与迁移能力。

# 本章要点

范式

## 生成式预训练

GPT 系列采用生成式预训练方式，将任务学习转变为预训练学习范式。

见 3.1。

架构

## 纯解码器 Transformer

核心架构基于纯解码器 Transformer，通过大规模预测下一个词元的任务进行训练。

见 3.2。

能力

## 零样本与迁移

GPT 模型在零样本任务上表现出色，并且能够轻松迁移到各种下游任务。

表征与泛化能力是迈向 AGI 的关键基石之一。

分词与生成

## BPE 与解码策略

BPE 用统计+规则构建子词词表；生成阶段用贪婪、采样、Top-$K$、Top-$P$、束搜索控制输出。

见 3.3、3.4。

**依据** 《大语言模型：训练、精调与对齐》第 3.5 节

# 训练与推理的可扩展性

语言模型范式的成功，不仅在于预训练阶段获得了强大的通用语言表征能力，更在于这一过程本身需要庞大的算力支持。

## 学习：表征获取

预训练通过下一个词元预测，从海量语料中获取通用表征。

规模越大，表征越丰富，困惑度持续下降。

## 搜索：推理决策

回答高难度选择题时，模型基于采样算法进行决策；即使面对复杂问题，仍有概率给出正确答案。

这一决策过程本质上可视为一种搜索行为。

- 通过增加计算资源（堆叠算力），模型能够显著提升这种搜索/推理的效果。
- 得益于在“学习”（表征获取）和“搜索”（推理决策）两方面极强的可扩展性，后续研究的核心方向必然聚焦于深化对这两个维度的探索。

**依据** 《大语言模型：训练、精调与对齐》第 3.5 节

# 语言模型的通用化

语言模型的成功也深刻影响了其他领域：许多算法通过融合语言模型的强大语义表征能力，显著提升了自身性能。

视觉-语言

**CLIP**

零样本图像分类任务。

视觉-语言

**BLIP**

图像理解与描述任务。

分割

**SAM**

图像分割任务。

语音

**wav2vec**

语音表征学习。

GPT-3 的成功促进了语言模型的发展，并涌现出越来越多高效的预训练模型架构，如 Llama、DeepSeek-V3、RWKV、Mamba 和 LLaDA，使语言预训练模型成为最活跃且最具影响力的研究领域之一。

# 3.6*

# 语言模型的压缩理论*

本节内容

**编码分布与无损压缩**

算术编码

似然最大化与通用编码

“压缩即智能”

从信息论看，语言模型的预测能力可以直接转化为无损压缩；最优压缩与最优预测是同一件事。

标 * 的节为选读内容，不影响后续章节的连贯性。

# 压缩与智能的关系

在信息论框架下，压缩通过最小化数据的编码长度实现，而智能体现在通过预测或生成来有效处理数据。

- 大语言模型能在大量文本数据中训练，学习如何有效地表示信息并进行预测，因而成为强大的压缩工具。
- 语言模型不仅预测下一个词或符号，还能把这些预测转化为压缩过程：以尽可能少的比特编码信息，实现无损压缩。
- 记号约定：来自有限符号集 $\mathcal{X}$、长度为 $n$ 的数据流 $\boldsymbol{x}_{1:n} := x_1 x_2 \cdots x_n \in \mathcal{X}^n$，空字符串记为 $\epsilon$，$sr$ 表示字符串 $s$ 与 $r$ 的拼接。

**编码分布 $p$**

概率模型

**算术编码**

区间逐步缩小

**交叉熵**

次优码长

**似然最大化**

等价于最小化码长

# 编码分布

编码分布 $p$ 是一系列概率质量函数 $p_n : \mathcal{X}^n \mapsto (0, 1]$，对任意 $n \in \mathbb{N}$ 满足相容约束：

$$\rho_n(\boldsymbol{x}_{1:n}) = \sum_{y \in \mathcal{X}} \rho_{n+1}(\boldsymbol{x}_{1:n} y)$$

基例为 $\rho_0(\epsilon) := 1$：把更长的字符串的边缘概率求和，正好回到较短的前缀，这是"可续写"的数学表述。

$$\rho(x_n \mid \boldsymbol{x}_{<n}) := \frac{\rho(\boldsymbol{x}_{1:n})}{\rho(\boldsymbol{x}_{<n})}$$

符号 $x_n$ 在给定先前数据 $\boldsymbol{x}_{<n}$ 时的条件概率，就是两个前缀概率之比。

$$\rho(\boldsymbol{x}_{1:n}) = \prod_{i=1}^{n} \rho(x_i \mid \boldsymbol{x}_{<i}) \quad \text{和} \quad \rho(\boldsymbol{x}_{j:k} \mid \boldsymbol{x}_{<j}) = \prod_{i=j}^{k} \rho(x_i \mid \boldsymbol{x}_{<i})$$

链式法则：整段序列的概率等于逐位置条件概率的连乘——这正是自回归语言模型的形式。

# 无损压缩的码长

无损压缩要把从编码分布 $p$ 采样的符号流编码为最小期望长度的比特流，同时保证原始序列可恢复。

$$c : \mathcal{X}^* \mapsto \{0,1\}^*$$

二进制源编码 $c$ 为每个可能的数据序列 $\boldsymbol{x}_{1:n}$ 分配一个长度为 $\ell_c(\boldsymbol{x}_{1:n})$ 比特的码字。

$$L := E_{\boldsymbol{x}\sim\rho}[\ell_c(\boldsymbol{x})]$$

目标是最小化每个序列的期望比特数：用更多比特编码稀有序列，用更少比特编码频繁序列。

$$L \geqslant H(\rho), \qquad H(\rho) := E_{\boldsymbol{x}\sim\rho}[-\log_2 \rho(\boldsymbol{x})]$$

香农的源编码定理描述了数据压缩的可能极限：任何编码的期望码长都不低于熵 $H(\rho)$。

**依据** 《大语言模型：训练、精调与对齐》第 3.6 节*

# 算术编码

给定编码分布 $\rho$ 和序列 $\boldsymbol{x}_{1:n}$，算术编码构造一个几乎最优长度的码。

$$-\lceil \log \rho(x_{1:n}) \rceil + 1 \quad \text{vs.} \quad -\log \rho(\boldsymbol{x}_{1:n})$$

左侧为算术编码的长度（假设无限精度），右侧为最优编码长度。

- 实际实现中，若受限于 $B$ 比特精度，则额外增加 $O(n2^{-B})$ 比特，这对 32 位或 64 位算术可以忽略。
- 算术编码把整段序列映射为 $[0, 1)$ 区间内的一个数，因此可以逼近 $\rho$ 下界所允许的最短码长。
- 它依赖的正是 $\rho$ 的条件概率 $\rho(y \mid \boldsymbol{x}_{<k})$——也就是语言模型每一步给出的分布。

**依据** 《大语言模型：训练、精调与对齐》第 3.6 节*

# 算术编码的计算过程

序列 $\boldsymbol{x}_{1:n}$ 的算术编码是区间 $[0,1)$ 内一个数 $\lambda$ 的二进制表示。初始区间 $I_0 = [0,1)$，每编码一个符号就缩小区间。

$$\tilde{I}_k(\boldsymbol{x}) := \left[ l_{k-1} + (u_{k-1} - l_{k-1}) \sum_{y<\boldsymbol{x}} \rho(y \mid \boldsymbol{x}_{<k}),\quad l_{k-1} + (u_{k-1} - l_{k-1}) \sum_{y\leqslant\boldsymbol{x}} \rho(y \mid \boldsymbol{x}_{<k}) \right]$$

步骤 $k > 0$（编码 $x_k$）时，把前一区间 $I_{k-1} = [l_{k-1}, u_{k-1})$ 划分为 $N$ 个子区间，每个对应 $\mathcal{X}$ 中一个字母；字母 $y$ 的子区间大小为 $(u_{k-1} - l_{k-1}) \cdot \rho(y \mid \boldsymbol{x}_{<k})$（假设 $\mathcal{X}$ 上存在严格顺序）。

- 为编码 $x_k$，选择其对应区间 $I_k = \tilde{I}_k(x_k)$：每编码一个符号，区间就按该符号的条件概率收窄一次。
- 最后在终止区间 $I_n$ 中选择具有最短二进制表示的 $\lambda$，用该二进制表示编码 $\boldsymbol{x}_{1:n}$。

**依据** 《大语言模型：训练、精调与对齐》第 3.6 节*

# 交叉熵与次优码长

实践中源分布 $\rho$ 通常未知，而是用参数化概率模型 $\hat{\rho}$ 估计。因此实际码字长度不是最优长度。

$$-\sum_{i=1}^{n}\log_2\hat{\rho}(x_i\mid \boldsymbol{x}_{<i}) \quad \text{vs.} \quad -\sum_{i=1}^{n}\log_2\rho(x_i\mid \boldsymbol{x}_{<i})$$

左侧为用 $\hat{\rho}$ 得到的码字长度，右侧为最优长度；两者的差就是模型估计不准所付出的比特。

$$H(\rho,\hat{\rho}) := E_{\boldsymbol{x}\sim\rho}\left[\sum_{i=1}^{n}-\log_2\hat{\rho}(x_i\mid \boldsymbol{x}_{<i})\right]$$

期望（次优）比特数即交叉熵。

- 最小化相对于 $\hat{\rho}$ 的交叉熵，就最小化了按 $\rho$ 分布的符号流的期望编码长度。
- 这等价于似然最大化——这就是用交叉熵/负对数似然训练语言模型的压缩解释。

**依据** 《大语言模型：训练、精调与对齐》第 3.6 节*

# 最优压缩与最优预测的等价性

前面讨论的是相对于固定分布采样的数据的最优编码。相对于**所有可计算采样分布**的通用最优源编码，可以把码长取为 $\boldsymbol{x}_{1:n}$ 的柯尔莫哥洛夫复杂度。

$$S(\boldsymbol{x}_{1:n}) = \sum_{q \in \mathcal{Q}} 2^{-\ell_c(q)} q(\boldsymbol{x}_{1:n})$$

Solomonoff 预测器：在图灵完备编程语言中，对所有可编程预测器 $q$ 做贝叶斯混合，权重为 $2^{-\ell_c(q)}$。

**结论** 由此，最优压缩等价于最优预测。

- “可编程预测器”就是候选的模型程序；越短的程序权重越大，这与“更小的描述长度代表更好的理解”一致。
- 这条线索把语言模型的训练目标（预测）与压缩目标（码长）统一起来。

**依据** 《大语言模型：训练、精调与对齐》第 3.6 节*

# 压缩与智能的关联

在通用人工智能研究中，有一种思潮认为：实现对有效信息的最大限度无损压缩，就是实现 AGI 的核心目标。

$$|D| = -\log P_f(D) + |f|$$

$P_f(D)$ 是使用模型 $f$ 计算数据 $D$ 的概率，$|f|$ 是实例化并训练模型 $f$ 所需的代码长度（约 100KB）。压缩数据 $D$ 等价于提升 LLM 对下一符号的预测能力。

- 该表达式不依赖于模型的规模，因此优化 LLM 的压缩能力实际上可以增强其泛化能力，使其高效表征训练数据之外的信息。
- 从翻译任务看：对完成任务的有效方法，最小描述长度代表对该任务的最大理解。
- 感知可视为两步：收集所有有用的感知信息；再通过强大的基础模型尽可能高效地压缩。
- 提升压缩能力的途径：更优的架构设计、扩展模型规模、工具使用、记忆检索、合成数据等，共同指向在尽可能广泛的观测空间内实现最优的信息压缩。

# 思考题与实践题

1 预训练模型在推理时会有重复生成（“复读机”现象），其原因是什么？

2 GPT-2 只保留语言头。若要做情感分类，提示应当如何构造，输出又如何映射为类别？

3 后置归一化为什么会让残差连接的信息逐层衰减？前置归一化为此付出了什么代价？

4 预测下一个词元时，注意力以上下文中的哪个词元作为检索？

5 GELU 中 $\Phi(x)$ 的作用是什么？$x \to -\infty$ 与 $x \to +\infty$ 时，$\mathbf{GELU}(x)$ 分别趋近于什么？

6 BPE 编码 “babcabc” 时，为什么不能因为词表里有 ‘abc’ 就直接匹配？合并表的顺序起了什么作用？

7 实践：实现一个 Top-$P$ 生成器，比较 $p = 0.5$、$0.9$、$1.0$ 时同一提示的输出差异。

8 实践：实现一个 BPE 编码器，在 “abcabcab” 上复现图 3.10–3.12 的词表、合并表与编码结果。